Claude Code Alternatives: A Comprehensive Survey of AI Coding Agents in 2026

Mick Cooney

12 October 2026

Table of Contents

1 Introduction **4**

2 Capability-led Selection and Adoption **5**
2.1 Choose a capability lane . . . 5
2.2 A staged adoption path . . . 7
2.3 Command contract for every example . . . 8

3 The AI Coding Agent Taxonomy **10**

4 Model Alternatives for Coding Agents **10**
4.1 Model, Provider, and Agent Harness . . . 11
4.2 What Matters for Agentic Coding . . . 11
4.3 Candidate Families Against the Claude Baseline . . . 12
4.4 Closed-Weight Models . . . 14
4.4.1 Anthropic / Claude Baseline . . . 14
4.4.2 OpenAI / GPT and Codex Models . . . 14
4.4.3 Google / Gemini . . . 15
4.4.4 xAI / Grok . . . 15
4.4.5 Other Vendors and Product-Specific Models . . . 15
4.5 Open-Weight Models . . . 16
4.5.1 Qwen . . . 16
4.5.2 DeepSeek . . . 16
4.5.3 Z.AI / GLM . . . 17
4.5.4 Moonshot / Kimi . . . 17
4.5.5 Mistral / Devstral . . . 18
4.5.6 OpenAI / gpt-oss . . . 18
4.5.7 Meta / Llama and Google / Gemma . . . 18
4.5.8 Other Families: MiniMax, NVIDIA Nemotron, IBM Granite, Microsoft Phi . . . 19
4.6 Deployment Fit: Workstation, Server, or Hosted Endpoint . . . 19
4.7 Evidence and Evaluation . . . 21

4.8 Choosing a Replacement . . . 22

5 CLI-Native Tools **23**
5.1 Aider . . . 23
5.2 OpenCode . . . 26
5.3 Goose (by Block) . . . 27
5.4 Gemini CLI (by Google) . . . 29
5.5 Plandex . . . 30
5.6 SWE-agent . . . 31
5.7 OpenAI Codex CLI . . . 32
5.8 Amp (Sourcegraph) . . . 33
5.9 Deprecated and Unverified Tools . . . 34
5.9.1 Mentat . . . 34
5.10 Qwen Code and Kimi CLI . . . 35

6 IDE Extensions and Hybrid Tools **35**
6.1 Cline (formerly Claude Dev) . . . 35
6.2 Continue.dev . . . 37

7 Dedicated AI IDEs **39**
7.1 Cursor . . . 39
7.2 Windsurf (now Devin Desktop, Cognition AI) . . . 40
7.3 Zed AI . . . 41
7.4 Google Antigravity . . . 42
7.5 AWS Kiro IDE and CLI . . . 43

8 Cloud and Web Platform Agents **44**
8.1 Grok Bot (xAI) . . . 44
8.2 OpenHands (formerly OpenDevin) . . . 46
8.3 Devin (by Cognition AI) . . . 47

9 Commercial AI Coding Assistants **48**
9.1 GitHub Copilot with Agent Mode . . . 48
9.2 Amazon Q Developer (formerly CodeWhisperer) — IDE transition to Kiro . . . 49
9.3 JetBrains AI Assistant . . . 50
9.4 Tabnine . . . 51
9.5 Supermaven — discontinued, folded into Cursor . . . 51

10 Cross-Cutting Analysis **52**
10.1 Provider Flexibility Analysis . . . 52
10.1.1 OpenRouter Compatibility . . . 52
10.1.2 Local LLM Support (Ollama / LM Studio) . . . 54
10.2 Feature Comparison Matrix . . . 56
10.3 Extensibility Deep Dive . . . 60
10.3.1 Claude Code's Extension Architecture . . . 60
10.3.2 How Alternatives Compare . . . 61
10.3.3 The AGENTS.md Open Standard . . . 62

10.3.4 What This Means for Migration . . . 62
10.4 Cost Analysis . . . 62
10.4.1 Defining "Heavy Usage" . . . 63
10.4.2 API Cost Estimates (Direct Provider, as of 7 October 2026) . . . 63
10.4.3 Per-Tool Cost Summary . . . 65
10.4.4 Cost Migration Strategy . . . 70

11 Recommendations **71**
11.1 If you want the closest CLI experience to Claude Code . . . 71
11.2 If you want the best IDE-integrated alternative . . . 71
11.3 If cost is the primary constraint . . . 72
11.4 If you need persistent cross-application or asynchronous work . . . 72
11.5 If MCP server investments are critical . . . 72
11.6 If you need extensibility similar to Claude Code's skills+hooks . . . 73
11.7 Migration strategy . . . 73

12 A Concrete Multi-Model Alternative Setup **74**
12.1 The recommended default: OpenCode plus explicit model tiers . . . 74
12.2 Provider accounts, credits, and data boundaries . . . 75
12.3 Rough cost planning . . . 81
12.4 Model allocation by task . . . 82
12.5 Setup: CLI-first . . . 84
12.6 Setup: agentic editor . . . 84
12.7 Repository configuration and security boundaries . . . 85
12.8 A staged rollout and evaluation loop . . . 86
12.9 What this setup does not solve . . . 87

13 Pi as an Additional Harness **87**
13.1 What Pi adds to a multi-harness setup . . . 88
13.2 Installation and a first controlled run . . . 89
13.3 Project context, settings, and extensions . . . 89
13.4 What Pi does not provide by default . . . 90
13.5 Sessions and cross-harness handoff . . . 91
13.6 How to evaluate Pi fairly . . . 91
13.7 Recommendation . . . 92

14 References **92**
14.1 Model Documentation and Licences . . . 92
14.2 Model Evaluations and Practitioner Reports . . . 93
14.3 Tool and Pricing Documentation . . . 93
14.4 Secondary Background . . . 94

1 Introduction

Claude Code has established itself as one of the most capable agentic coding tools available: it runs in the terminal and supported integrated development environments (IDEs), takes high-level natural-language instructions, autonomously edits multiple files, executes shell commands, runs tests, and iterates until the task is done. Its extensibility system — skills, hooks, and Model Context Protocol (MCP) server support — allows deep customisation of its workflow. For heavy users, a Max plan (**$100/month for Max 5x, $200/month for Max 20x**) can be good value relative to metered API pricing, but it is not unlimited: Max has a five-hour session limit and a separate weekly limit, and Anthropic may apply additional caps. Limits are shared across Claude, Claude Code, and Claude Desktop. The plan landscape for AI tools changes rapidly, and a prudent engineer should understand the full landscape of alternatives before needing them.

This document surveys the landscape of AI coding agents available as of **7 October 2026**: open-source CLI tools, IDE extensions, dedicated AI IDEs, cloud platform agents, and commercial assistants. For each, it covers architecture, provider flexibility, MCP/extensibility support, realistic cost, and the model capabilities that determine whether a tool can replace a Claude Code workflow. Choosing the agent harness and choosing the model are related but separate decisions: the same model can behave differently under different context, editing, permission, and recovery machinery.

A note on methodology and provenance: This document was originally drafted from an AI model's training-data snapshot (accurate as of approximately August 2025), then refreshed via live web research in July, August, and September 2026. The August 31 pass rechecked the most volatile claims against first-party product pages, including Claude and Codex limits, Amp's subscription and Orb pricing, Cline's terminal/plugin/hook support, Goose's current repository, Kiro's unified IDE/CLI/Web architecture, Gemini CLI's transition status, and DeepSeek pricing. The September 9 pass added Grok Bot and checked its launch, architecture, controls, privacy requirements, and current access against first-party xAI pages. The September 16 pass rechecked the current Claude, OpenAI, Gemini, DeepSeek, and Z.AI model lineups and added practitioner reports about model routing and agent-harness reliability. Dated facts carry a date at the point where the distinction matters, with corresponding sources in the References section.

Freshness note: In this field, some sections can age in weeks, not quarters. Treat pricing, benchmark rankings, and model-version statements as snapshots tied to their stated dates.

How to read this document: If you want the fastest path to a conclusion, start with

Capability-led selection and adoption for the short decision, staged rollout, and command-safety contract. Then use Model Alternatives for Coding Agents, the Feature Comparison Matrix, Provider Flexibility Analysis, and Detailed recommendations as evidence. The deep-dive profiles, concrete multi-model setup, and Pi section remain reference material for an evaluation you can reproduce.

This survey covers tools that were verifiable and actively maintained as of October 2026. It does not cover tools no longer in active development, purely GUI-based editors with no API or CLI surface, or general-purpose LLM interfaces that happen to accept code. The model section is a coding- and agent-workflow survey, not a catalogue of every chat, image, embedding, or research model. Cloud IDE platforms (Replit, Gitpod, etc.) are out of scope unless they offer a dedicated coding-agent mode. Where a claim could not be verified, it is flagged.

Verified against primary sources for all sections; unverified claims are flagged inline.

Assume a working knowledge of version control, shell commands, software-development workflows, and API credentials. This is a comparative survey and evaluation guide, not a product endorsement, security approval, benchmark result, or substitute for testing an agent against your own repository and data-handling requirements.

---

2 Capability-led Selection and Adoption

The shortest useful answer is to choose the operating boundary first, then choose a model inside it. Do not start by translating Claude model names into another vendor's names: the harness, provider, context policy, tools, permissions, and recovery loop are part of the system you are evaluating.

2.1 Choose a capability lane

Use the first row that matches the constraint that matters most. The named tools are starting points, not claims of universal superiority; the dated profiles and matrix below explain the evidence and the exceptions.

Primary requirement	Start here	Why this is the default	Main reason to choose another lane
Terminal-native, multi-provider work with MCP	**OpenCode** with an approved provider; keep **Aider** as the small git-native fallback	OpenCode combines a terminal surface, broad provider choice, Language Server Protocol (LSP), MCP, and multi-session work; Aider is simpler and automatically git-oriented	Choose Aider when minimalism and predictable git edits matter more than MCP; choose Goose when on-device execution and MCP portability are the priority
Editor-first Plan/Act work	**Cline**; consider **Continue.dev** for local inference or **Cursor** for a polished commercial IDE	Cline provides a model-agnostic editor workflow with reviewable planning, browser/MCP access, and a CLI; Continue prioritises local models	Choose a CLI when scripting or headless operation is primary; choose Cursor only when its hosted, VS Code-derived boundary is acceptable
Sensitive or offline routine work	**A local model through OpenCode, Continue.dev, Aider, or Pi**	The inference request can remain on the workstation when the full tool and extension path is audited	A local model does not protect data sent by telemetry, remote MCP, extensions, model downloads, or editor integrations
Long-running cloud or cross-application work	**OpenHands** for an open/self-hostable coding stack; **Devin** or **Grok Bot** when a managed cloud computer is acceptable	These products address asynchronous execution and broader computer or repository boundaries	Use a local terminal harness when local source control, data locality, or interactive approval is the central requirement

Primary requirement	Start here	Why this is the default	Main reason to choose another lane
Small, inspectable experimental harness	**Pi** beside OpenCode, not underneath it	Pi makes model switching, session state, and TypeScript-level workflow customisation explicit	Use OpenCode or Cline when MCP, plan/approval features, or a richer default tool surface is required

For model allocation, begin with four roles rather than four permanent model names: a frontier route for hard diagnosis, a normal implementation route, a fast/bounded route, and an independent-review route. The candidate-family shortlist supplies dated examples. The concrete setup later turns those roles into OpenCode/OpenRouter, direct-provider, local, Aider, Cline, and Pi configurations. A model choice never grants additional shell, file, network, credential, or merge authority.

2.2 A staged adoption path

Treat migration as an evaluation programme. Keep the existing Claude workflow as the baseline and promote alternatives only when the complete harness/provider/model configuration passes the same acceptance checks.

1. **Define the boundary and baseline.** Record the repository revision, task contract, Claude harness and model, permissions, provider, context policy, acceptance checks, cost, latency, retries, and human correction. Decide whether source may leave the workstation. Create a disposable branch or worktree for any write-capable trial.
2. **Run a read-only smoke test.** Verify installation, version, authentication, model listing, provider route, and repository instructions with an inventory or test-plan request that must not edit files. Stop on an unknown endpoint, unexpected credential path, or data-boundary mismatch.
3. **Pilot the lanes.** Run roughly 20–30 representative tasks: bounded support, ordinary implementation, hard diagnosis, documentation, a failing test, a multi-file change, and an MCP/tool task where relevant. Compare accepted changes, missed defects, false positives, recovery, review burden, elapsed time, tool failures, and spend—not just whether the final response says it succeeded.
4. **Add diversity deliberately.** Use a different provider or local route for high-risk

review when the data policy permits it. Add Pi as a harness comparison using the same task contract, revision, permissions, model tuple, and checks; record any extension or wrapper as part of Pi's identity. Do not call two sessions independent merely because their model names differ.

5. **Promote narrowly and re-evaluate on change.** Give a candidate only the role and permissions it passed. Re-run the pilot when the provider, model ID, quantisation, context setting, harness version, project instructions, or retention terms change. Human review and CI remain the merge gate.

The recommended starting configuration is therefore **OpenCode plus a scoped OpenRouter project/key**, with an explicit hard/normal/bounded/review allocation and a local route for data that must stay on the workstation. Keep Aider as the minimal fallback, Cline as the editor-first option, and Pi as the inspectable harness experiment. Prefer direct providers or self-hosting when they provide a required regional, contractual, or data boundary; OpenRouter is a convenient routing layer, not a privacy guarantee. The provider-specific retention distinctions later in this document are part of the decision, not footnotes.

2.3 Command contract for every example

The installation, authentication, configuration, and launch commands throughout this document are illustrative recipes. Before retaining or running any one of them, apply the following contract to that command and record the result in the pilot notes. A command is not authorised merely because it appears in a profile.

Requirement	What the operator must establish before running the command
Prerequisites	The supported operating system/runtime, package manager or container engine, repository/worktree, version pin, network access, and any required service account or credits. Verify the live project documentation because these facts are dated.

Requirement	What the operator must establish before running the command
Authority and credentials	Which user, branch/worktree, shell permissions, provider account, API key, OAuth session, MCP token, or local endpoint the command uses. Scope keys and permissions to the pilot; never put secrets in tracked configuration or command captures.
Verification	A non-destructive version/status/model/provider check, then the expected file, process, diff, test, or API result. Record the exact harness, model, provider, and settings; a successful login does not prove tool-call or retention compatibility.
Rollback	How to stop the process, revoke or rotate credentials, remove only the installed/configured component, discard or reset the disposable worktree, restore configuration, and undo generated changes. Do not use a broad reset or deletion command when a targeted reversal is sufficient.

For commands that install software or start a server, verify the package/source and release before execution. For commands that export credentials, prefer a password manager, the harness credential store, or a non-committed environment file; clear captured secrets and rotate exposed keys. For commands that grant write or shell access, begin read-only, use least privilege, require approval for destructive or external actions, and inspect the diff and test results before integration. The individual setup sections below add provider- and product-specific checks; if a section cannot satisfy one of these four fields, treat the command as unverified rather than silently copying it.

---

3 The AI Coding Agent Taxonomy

Before comparing tools, it helps to understand the four distinct categories that have emerged. This taxonomy comes from Artificial Analysis's coding agent classification (verified):

CLI Tools run entirely in the terminal. They take instructions, edit files, run commands, and loop autonomously. This is the category Claude Code belongs to. Other members: Aider, OpenCode, Goose, Gemini CLI (legacy/enterprise path), Antigravity CLI, Plandex, SWE-agent (Software Engineering agent), Qwen Code, Kimi CLI, OpenAI Codex CLI, Amp (hybrid local/cloud), and GitHub Copilot CLI (distinct from the IDE-based Copilot covered under IDE Extensions below).

IDE Extensions augment an existing editor (primarily VS Code or JetBrains). They have full access to the editor's language server, refactoring tools, and UI, but are less suitable for scripted or headless workflows. Members: Cline, Continue.dev, GitHub Copilot, Amazon Q Developer, Tabnine, JetBrains AI Assistant.

Dedicated AI IDEs are entire editors rebuilt around AI-first workflows. They typically fork VS Code and add deeper AI integration than an extension permits (Zed is the exception, built from scratch in Rust). Members: Cursor, Windsurf (now Devin Desktop, Cognition AI), Zed AI, Amazon Web Services (AWS) Kiro IDE, Google Antigravity 2.0.

Cloud Platform Agents run primarily in the cloud or a sandboxed environment (Docker). They expose a web or app-based interface and are designed for longer-running autonomous tasks, often with their own execution environments. Members: Grok Bot, OpenHands, Devin, Manus, Jules, Genie. Grok Bot sits at the broadest edge of this category: it is a persistent cross-application computer-use agent that can do coding work, not a coding-specific terminal agent.

A Claude Code user primarily cares about the CLI tools category, but the IDE and cloud categories contain tools capable enough to be worth understanding as alternatives — especially if your workflow includes time in an editor.

---

4 Model Alternatives for Coding Agents

Choosing an agent and choosing its model are separate decisions. You can preserve a familiar editing workflow while changing the inference endpoint, or keep the model while changing how the agent gathers context and executes work. This section is the model survey for the

7 October 2026 snapshot; the tool profiles that follow describe the surrounding products. The objective is to identify candidates for your workload, not to translate every Claude name into another vendor's tier.

4.1 Model, Provider, and Agent Harness

The **model** is the learned set of numerical weights; a **checkpoint** is a particular saved version of those weights. Training, architecture, and post-training determine what it can infer and the kinds of responses it can produce. A family name alone does not specify the checkpoint, reasoning setting, input modalities, or serving configuration you will actually use.

The **inference provider** runs the model and returns its output. A gateway such as OpenRouter can add another routing layer between your agent and that provider. Providers can differ in model revisions, context limits, supported request fields, availability, retention, and inference speed. An API-compatible endpoint can accept a request while interpreting some fields differently; matching the API shape is not proof of equivalent behaviour.

The **agent harness** supplies repository context, presents tool definitions, applies edits, enforces permissions, executes commands, and handles retries, compaction, and recovery. The model proposes a tool call; the harness or a provider service executes it. Shell access, apply-patch execution, and Model Context Protocol (MCP) connections are harness or product capabilities, not facilities embedded in model weights. A model must still generate valid arguments and interpret the results correctly. Diagnose failures at the right layer before replacing the model.

For example, if a patch targets an outdated file fragment, the model may have misunderstood the code, the harness may have retained stale context, or the edit adapter may have rejected an otherwise useful change. Record the attempted edit and tool response: a final answer saying "fixed" does not identify which layer failed.

4.2 What Matters for Agentic Coding

Start with **edit correctness**: does the change satisfy the task, preserve unrelated behaviour, and respect repository conventions? Test **tool-call validity** separately: valid schemas, real paths, appropriate commands, and correct handling of tool results. Then test **multi-step recovery** by giving the agent a failed build, rejected patch, missing dependency, or interrupted session. An agent that repeatedly applies the same failing fix is not reliable merely because its first code sample was convincing.

Effective context is the material the model can use coherently, not the maximum token

count an endpoint accepts. A token is a unit of encoded text or other input; context includes instructions, source, tool results, and conversation history. Test whether constraints survive a long session and its compaction into a shorter summary. **Modality** matters when the task depends on screenshots or diagrams: the checkpoint, provider, and harness must all support the relevant input. Text-only models can still work with extracted text, but cannot inspect the original visual evidence.

Measure **latency and cost per accepted task**, including retries and human correction. A low token price can lose its advantage after repeated failed edits. **Reasoning controls** change how much inference work the model spends before responding; effort names and budgets are not comparable units across vendors. Test settings with a fixed task budget rather than assuming that every "high" setting behaves alike.

Operational privacy follows the entire data path. A local terminal can send source to a hosted model, and a local model can sit inside a harness that sends telemetry or invokes remote tools. Decide whether repository contents may leave the workstation, an internal server, or an approved hosted boundary. Check the provider and gateway's applicable retention and access controls as well as the harness configuration.

Use four operational roles: **bounded support** for inventory, summaries, log triage, and small constrained transformations; **normal implementation** for the edit-test loop; **hard diagnosis** for uncertain causes and repeated failures; and **independent review** for checking a proposed change against evidence. These roles specify work and authority. They are not universal Haiku, Sonnet, Opus, or Fable equivalents.

4.3 Candidate Families Against the Claude Baseline

Use this **7 October 2026** shortlist to decide what to test against your existing Claude configuration. Rows select representative checkpoints, not equivalent vendor tiers. Coding and recovery suggestions are **qualitative synthesis**; context, input support, and prices are **documented specifications**, not measured task performance. The later evaluation discussion separates benchmark results from practitioner reports. The profiles below explain each family's qualifications.

Price pairs mean **USD per million uncached input / output tokens**, for the named direct API and standard service unless stated otherwise. They are cost anchors, not subscription prices or cost per completed task. Context figures are advertised limits; they do not measure how well instructions survive a long session.

Family / example	Coding / recovery focus	Context / visual input	Cost / deployment
Claude baseline	Pin model and effort; measure edits, calls, and recovery	Sonnet 5.5; images	Hosted; verify current model pricing before comparison
GPT / Codex models	Reasoning and function calls; test difficult fixes	GPT-5.6/6: 1.05M; images	Hosted; Terra $2 / $12, short context
Gemini 3.8 Flash	Tool-driven implementation; test long-session recovery	~1M input; images, video, audio, PDFs	Hosted; $0.75 / $3.75, promotional
Grok 4.6	Reasoning and function calls; test edit-loop reliability	500K; images	Hosted; $2 / $6 at up to 200K context
Qwen / Qwen3-Coder	Coding-specialised tools; test patch correctness	Coder: 256K native; vision varies by checkpoint	Open; large Coder needs servers; smaller variants offer local experiments
DeepSeek V4.1 Flash	Thinking and tool calls; test failed-fix recovery	1M; images	Open/server or API; $0.30 / $1.20 peak; half off-peak
Z.AI / GLM-5	Engineering and tool use; test planning through execution	200K; text only for GLM-5	Open/server or hosted; coding-plan quotas differ from API billing
Kimi K3	Long implementation and visual tasks; test reasoning-state continuity	1M; images and video	Open; hosted or substantial cluster; release-specific licence
Devstral Small 2	Code-focused multi-file edits; test recovery after rejected patches	256K; images	Open, Apache 2.0; 24B local candidate, with runtime/context costs
gpt-oss-20b / 120b	General reasoning and calls; test tool-format compatibility	~131K; text only	Open, Apache 2.0; 20b local candidate; 120b needs much more memory

Cost and evidence limits: Gemini's quoted promotion runs through 31 December 2026;

OpenAI and Grok charge more for long-context requests. Caching, reasoning output, provider tools, and retries change the bill. Self-hosting replaces API charges with hardware, energy, and serving work; open weights do not imply lower total cost or unrestricted licensing. Compare accepted changes, elapsed time, and correction effort with the same Claude baseline before claiming parity or savings. This table supplies no measured ranking, guaranteed workstation fit, or proven Fable substitute.

4.4 Closed-Weight Models

Closed-weight models are available through services without downloadable weights for independent deployment. Their model documentation establishes advertised capabilities; your subscription or endpoint determines actual access. Keep a known Claude configuration as a baseline when testing alternatives.

4.4.1 Anthropic / Claude Baseline

This October refresh uses Anthropic's current Sonnet 5.5 and Opus 5.5 releases as the relevant Claude baseline; check the Anthropic model overview for the exact catalogue and lifecycle status before reproducing a comparison. Anthropic's capability positioning is a vendor claim, not a fixed task classification. Record the exact model, effort, context, and price used in your baseline.

A replacement should be compared with the Claude configuration you actually use on the same repository tasks. Reproducing a result from Claude Code may require preserving context selection and edit tooling as well as changing the model. No evidence presented here establishes a proven Fable equivalent.

4.4.2 OpenAI / GPT and Codex Models

OpenAI's hosted catalogue includes GPT-6 Astra and GPT-5.6 Sol, Terra, and Luna. OpenAI positions Astra for its hardest reasoning and coding work, Sol for complex professional work, Terra for balancing capability and cost, and Luna for cost-sensitive workloads. These are useful shortlist distinctions; they do not establish equivalence to Anthropic tiers. Check each model's supported modalities, context, function calling, and reasoning settings in the OpenAI model catalogue. Treat OpenAI's comparative coding scores as vendor evaluations.

Codex is also a product and access path, not a synonym for one immutable model. Codex-branded model releases and the models available inside Codex can change independently of API availability. A Codex subscription, hosted GPT API usage, and a self-hosted checkpoint are different arrangements; see the current product model documentation and the

Codex CLI profile for the product layer. An API listing shell, apply-patch, or MCP support describes integration with those tools, not a model executing them unaided.

OpenAI's **gpt-oss-20b and gpt-oss-120b** are a separate open-weight family, covered below. They are not downloadable versions of GPT-5.6 or GPT-6, and an open-source Codex client does not make the hosted GPT weights open.

4.4.3 Google / Gemini

Google's catalogue spans Pro, Flash, and Flash-Lite models. In this snapshot, Gemini 3.1 Pro remains a preview option, while Gemini 3.8 Flash is listed as stable and aimed at long software-engineering and agent tasks; Gemini 3.5 Flash-Lite targets throughput. Confirm lifecycle status and input support in the Gemini model catalogue, rather than treating "Pro" as an automatic coding winner.

Gemini merits evaluation when code must be interpreted alongside screenshots, documents, or other supported media. Large advertised context windows make repository-scale experiments possible, but do not establish constraint retention or recovery quality. Test those directly. Gemini CLI and Antigravity access, tools, and quotas belong to their tool profiles; neither is a property of the underlying Gemini checkpoint.

4.4.4 xAI / Grok

Grok belongs in the model shortlist independently of Grok Bot. xAI's Grok 4.6 documentation describes a hosted frontier model for coding and agentic tasks with function calling, structured output, reasoning, text/image input, and a 500K context window. Those specifications justify testing function-call correctness and recovery in your chosen coding harness; xAI's quality claims do not establish performance on your repository.

Check the exact endpoint's reasoning controls, image support, and context limits. Search services and a persistent cloud computer belong to provider tools or products. In particular, access to a Grok API model does not supply Grok Bot's execution environment. Earlier downloadable Grok releases also do not establish that the current hosted model has downloadable weights.

4.4.5 Other Vendors and Product-Specific Models

A managed coding product may select private models, fine-tunes, or several vendors behind its interface. Evaluate the resulting product if it is useful, but do not infer a portable model choice from its brand name or a successful demo. Without a documented checkpoint or

selectable endpoint, its results cannot answer whether the same behaviour will transfer to another harness. The existing commercial-tool profiles cover those product decisions.

4.5 Open-Weight Models

Open weights means that you can obtain the model's learned parameters and run them yourself under the release's licence. It does not necessarily mean unrestricted use, published training data, or a fully reproducible training process. Check the licence for the exact checkpoint. Many open-weight families also have hosted endpoints: downloadable weights and local deployment are separate facts.

Several families use a **mixture-of-experts (MoE)** architecture: a router activates only a subset of expert networks for each token. Active parameters help explain computation per token; the full collection of weights still needs storage and an effective way to reach the processors. **Quantisation** stores weights or intermediate values at reduced numerical precision, lowering memory demand and sometimes compute cost, with accuracy and runtime-support tradeoffs. A quantised checkpoint is a configuration to evaluate, not a guarantee of unchanged tool-use reliability.

4.5.1 Qwen

Qwen includes coding-specific and general-purpose models. Qwen3-Coder explicitly targets multi-turn software-engineering work with tool feedback. Its documented 480B-total/35B-active variant has 256K native context and an extended-context option; Qwen's reported coding results are vendor evaluations. That variant belongs on a server or hosted-endpoint shortlist, not a conventional single-GPU workstation shortlist. See the Qwen3-Coder release.

For a workstation experiment, the existing Qwen3.8-27B model card documents a general multimodal model with reasoning controls. It is not the same checkpoint or specialisation as Qwen3-Coder. A quantised 27B model is a plausible 24GB-class experiment at controlled context, subject to runtime overhead. Smaller Qwen checkpoints offer more memory margin; verify their own tool format and coding results. Qwen Code is the harness discussed later, not a capability inherited by every Qwen model.

4.5.2 DeepSeek

DeepSeek V4.1 Flash is available as an open-weight release as well as through the hosted `deepseek-flash` endpoint. The model card describes a multimodal MoE with a 552B-parameter backbone plus additional components, long context, and controllable reasoning.

"Flash" does not make this a workstation model: low active compute does not remove its large weight-storage requirement.

Use the DeepSeek API documentation to identify the hosted route and current limits, and the checkpoint's deployment instructions for self-hosting. The two routes need separate validation. Published coding and terminal scores are vendor results with specified harnesses; they do not prove recovery quality in yours. Smaller DeepSeek-derived or distilled checkpoints are separate models, not compact deployments of the full V4.1 system, and need their own evaluations.

4.5.3 Z.AI / GLM

Z.AI positions GLM-5 for agentic engineering, including planning, refactoring, and debugging. Its documented 744B total parameters versus 40B active make the deployment distinction concrete: this is a server-scale model despite its much smaller active computation. The published 200K context window is a specification; completion rates and long-task reliability remain evaluation questions.

The GLM-5.3-Flash announcement, dated September 16, describes native multimodal input and reduced inference requirements, but still lists 320B total parameters. Treat its performance comparisons as launch claims and verify checkpoint availability, serving support, and tool parsing before adoption. Smaller GLM variants may fit local experiments; neither the GLM name nor a coding-plan subscription establishes that a particular model fits your workstation.

4.5.4 Moonshot / Kimi

Moonshot's Kimi K3 technical account describes a 2.8-trillion-parameter model with native vision and a million-token context window; released weights have their own model card. Its scale makes it a hosted-endpoint or substantial server-cluster candidate. Moonshot recommends large accelerator deployments, so it is not a practical workstation replacement even with reduced-precision weights.

Kimi is relevant to long implementation sessions and visually informed work such as frontend debugging. Test whether the selected endpoint preserves reasoning state and tool results across turns, and whether its reasoning mode meets your latency budget. The K3 developer guide describes the API contract; launch demos and vendor benchmarks do not establish independent-review accuracy. Kimi CLI remains a separate harness choice.

4.5.5 Mistral / Devstral

Devstral is Mistral's coding-agent family, trained for repository exploration and multi-file software-engineering work. Its smaller downloadable variants offer a more realistic workstation experiment than the largest MoE families: Mistral's Devstral release describes a 24B small model as well as the larger model. Quantisation, context allocation, and the serving stack still determine whether your machine can sustain an agent loop.

Separate those checkpoints from hosted lifecycle status. Mistral currently marks the Devstral 2 endpoint deprecated and points new integrations to Mistral Medium 3.5, a multimodal model optimised for coding and agentic use with 256K context and open weights under a Modified MIT licence. That does not invalidate an existing local Devstral checkpoint, nor establish that Medium fits the same hardware. Evaluate the exact release, licence, and deployment instructions.

4.5.6 OpenAI / gpt-oss

OpenAI gpt-oss-20b and gpt-oss-120b are text-only open-weight reasoning models under Apache 2.0. They support configurable reasoning effort and function-call generation, but are general reasoning models rather than downloadable versions of the hosted Codex/GPT coding stack. The names round their total parameter counts: OpenAI documents about 21B/3.6B total/active for the smaller model and 117B/5.1B for the larger.

OpenAI's local Ollama guide describes supplied MXFP4 reduced-precision weights and a 16GB-or-more memory starting point for 20b; the 120b model is an approximately 80GB accelerator-class candidate with room depending on runtime and context. These are deployment starting points, not measured capacity for your workload. Preserve the required Harmony conversation format, including reasoning and tool boundaries, through the serving adapter. Test malformed calls, long-context recovery, and reasoning latency before giving either model write authority. Screenshots require a separate visual-processing path.

4.5.7 Meta / Llama and Google / Gemma

Llama offers downloadable general-purpose checkpoints with an established deployment ecosystem, but the family name does not imply code specialisation. Meta's model repository separates releases and their prompt formats; its full-precision Llama 4 examples require multiple GPUs. Smaller releases can be useful local baselines for summaries, retrieval over code, and bounded changes. Do not transfer results between generations, base and instruction-tuned checkpoints, or third-party coding fine-tunes.

Google's Gemma overview likewise describes a general-purpose open-weight family, distinct from hosted Gemini. Its current catalogue includes small deployment-oriented models and larger dense and MoE options, with modality varying by checkpoint. Gemma is useful when local memory limits or visual input drive the experiment, but a supported function-call format does not establish dependable autonomous coding. Check the exact release's licence, input support, and edit-test behaviour before moving beyond bounded support.

4.5.8 Other Families: MiniMax, NVIDIA Nemotron, IBM Granite, Microsoft Phi

MiniMax's coding and agent releases merit a hosted or server evaluation; their sparse active computation can obscure large total weights. NVIDIA Nemotron spans general reasoning and agent workloads across different deployment sizes. Its optimised runtime ecosystem may matter to an existing NVIDIA installation, but the family is not exclusively code-specialised and its larger variants are not workstation defaults.

IBM Granite Code provides explicitly code-oriented checkpoints; distinguish code completion and instruction variants from general Granite models when evaluating an agent loop. Microsoft Phi offers small general instruction/reasoning models suitable for constrained local experiments. For these smaller models, start with extraction, summaries, and tightly scoped edits. Compact size alone says nothing about reliable multi-file recovery.

4.6 Deployment Fit: Workstation, Server, or Hosted Endpoint

Size the deployment for the actual checkpoint and workload. GPU video memory (VRAM), shared system memory, and memory spread across several devices are not interchangeable performance budgets. CPU offload can make a model load while slowing every agent step; multiple GPUs require runtime support and sufficiently fast communication. The classes below are starting points for testing, not guaranteed fits.

Available accelerator memory	Candidate deployment class	Main constraint to measure
~16GB class	Small quantised Qwen, Gemma, Llama, Phi, or Granite checkpoints; gpt-oss-20b at the lower bound with a supported runtime	This is a 16GB-or-more starting point, not a sub-16GB guarantee; weight overhead and context cache can exhaust the margin

Available accelerator memory	Candidate deployment class	Main constraint to measure
24GB	Quantised models around 20–30B, such as smaller Devstral or Qwen3.8-27B; gpt-oss-20b with more headroom	Long context and simultaneous sessions may consume the margin
~80GB	gpt-oss-120b or selected quantised larger dense models	The advertised fit may leave insufficient room for your context or serving batch
Multi-GPU / server	Large Qwen3-Coder, DeepSeek, GLM, Kimi, and other large MoE checkpoints	Total weight storage, interconnect bandwidth, runtime support, and concurrency
Hosted endpoint	Any offered model that meets your data-boundary policy	Provider limits, queueing, request compatibility, and retention

The **key/value (KV) cache** stores attention state for previously processed tokens. Its size depends on architecture, precision, sequence length, and simultaneous sequences. Full-attention, sliding-window, and compressed-cache architectures have different requirements. Active parameters alone therefore cannot determine memory capacity; neither can a downloaded weight-file size.

For a small worked example, assume a hypothetical dense 14-billion-parameter model, ideal four-bit weight storage, 1 GiB of additional weight/quantisation overhead, and a 2 GiB runtime reserve. Assume full attention with 32 layers, eight KV heads per layer, 128 values per head, two-byte cache values, one sequence, and no cache sharing or compression. GiB here means 2^30 bytes. At 8,192 resident tokens, including generated tokens, the estimate is:

```
Weight payload = 14,000,000,000 × 4 / 8 bytes ~= 6.52 GiB
KV cache       = 32 × 8,192 × 2 (key and value) × 8 × 128 × 2 bytes
               = 1 GiB
Total          ~= 6.52 + 1 + 2 + 1 = 10.52 GiB
```

At 32,768 tokens, that cache becomes 4 GiB and the estimate rises to 13.52 GiB. Two independent sequences of that length need about 8 GiB of cache, bringing the estimate to 17.52 GiB before any extra batching overhead: too much for a 16 GiB budget. This is capacity arithmetic, not a benchmark or a prediction for the named models above. Measure actual peak allocation and latency. Hardware purchase, electricity, and administration costs belong in Cost Analysis.

4.7 Evidence and Evaluation

Keep four evidence types separate. **Vendor specifications and claims** establish intended formats, supported settings, licences, and published limits; vendor benchmark results show performance under that vendor's evaluation conditions. **Common-harness snapshots** reduce some comparison differences but still depend on tasks, budgets, prompts, and model adapters. **Practitioner reports** reveal failure modes worth reproducing, not population-level success rates. **Local deployment measurements** establish whether your selected quantisation, runtime, hardware, and concurrency meet your requirements.

As one common-harness snapshot, the DeepSWE v1.1 leaderboard labels its update 3 September 2026 and reports 113 long-horizon tasks using `mini-swe-agent`. Selected rows:

Model and setting	Resolution rate	Average cost	Notes / scope
GPT-6 Astra, xhigh	74% ±3%	$6.52/task	xhigh setting
Gemini 3.8 Flash, high	74% ±1%	$2.36/task	166 average agent steps
Claude Opus 5, max	74% ±4%	$11.84/task	max setting
Claude Fable 5, xhigh	70% ±3%	$13.41/task	Fable 5, not 5.1
GPT-5.6 Luna, max	67% ±4%	$0.61/task	No interactive-latency measurement
DeepSeek V4 Pro, max	63% ±6%	$1.67/task	Not V4.1 Flash

The uncertainty bands are reproduced as displayed. Overlap, differing effort settings, and benchmark-specific cost accounting preclude a universal ordering. This snapshot does not establish permission safety or performance in another harness.

The linked practitioner material includes Codex long-task stalls, OpenCode context-coherence concerns, and requests for Claude routing. Use those reports to design interruption, context, and escalation tests. They do not establish that one vendor fails more often. Community comparisons in the References are similarly leads for evaluation, not a substitute for controlled runs.

For a local comparison, start with 20–30 representative repository tasks across the four roles, including tasks with known acceptance criteria and deliberately failing commands or patches. Run each candidate from the same initial revision under the intended permissions and budget. Record the checkpoint or API identifier, provider, harness version, reasoning setting, context policy, and, when self-hosting, quantisation and runtime. Repeat ambiguous cases; do not hide retries behind a final success rate.

Measure accepted task outcomes, regressions, invalid calls, recovery attempts, elapsed time, human correction time, and cost. For local serving, also record peak memory, time to first token, generation speed, and failures at the intended context and concurrency. This survey does not supply measured workstation results; the capacity example above is explicitly an estimate.

4.8 Choosing a Replacement

Start with your deployment boundary and the consequences of an incorrect autonomous action, then shortlist models for a task role. The table supplies experiments, not winners. Keep the Claude baseline where its use is permitted, and compare actual accepted changes rather than the confidence of the final explanation.

Task role	Autonomy risk	Deployment boundary and evaluation shortlist	What to test
Bounded support	Low when read-only; rises with writes	Approved hosted: Luna, Gemini Flash-Lite, DeepSeek Flash; workstation: smaller Qwen, Gemma, Phi, or gpt-oss-20b	Extraction accuracy, constrained edit correctness, valid calls, response latency
Normal implementation	Moderate; changes multiple files	Approved hosted: Terra, Gemini Pro/Flash, Grok; self-hosted: Qwen3-Coder or Devstral sized to available hardware	Edit-test completion, unrelated-change avoidance, rejected-patch recovery
Hard diagnosis	High when uncertain conclusions trigger broad changes	Approved hosted: Astra, Sol, Claude baseline; hosted open-weight or internal server: GLM, Kimi, DeepSeek	Evidence-backed root cause, regression tests, recovery from failed hypotheses, stopping within budget

Task role	Autonomy risk	Deployment boundary and evaluation shortlist	What to test
Independent review	Read-only, but false assurance can conceal defects	A different family from the implementer, through a boundary permitted for the diff and source	Reproducible findings, missed defects, false positives, and whether evidence supports approval

Routing is a **policy to validate on your repository**. One starting policy is to allow bounded support without writes, assign ordinary changes to a tested implementation configuration, and escalate repeated failures or uncertain causes to diagnosis. Define an escalation trigger, such as two failed repair attempts, and a total time or spend limit. A more capable model does not automatically receive broader permissions.

For review, provide the task contract, diff, relevant source, test results, and unresolved questions. A different family may expose different assumptions, but does not guarantee statistical independence or detect every shared blind spot. Keep review findings tied to reproducible evidence. The model survey identifies what to test; Provider Flexibility Analysis and the tool profiles establish which harnesses can deliver the chosen policy.

---

5 CLI-Native Tools

5.1 Aider

What it is: Aider is the most mature and architecturally similar open-source alternative to Claude Code. It is a terminal-based pair-programming tool that works directly with your existing git repository, takes natural-language instructions, autonomously edits multiple files, and commits changes.

Author and licence: Created by Paul Gauthier. Apache-2.0 licence. Open source at `github.com/Aider-AI/aider`.

Community size: Roughly mid-40k to low-50k GitHub stars (late-July 2026 snapshot). This is among the largest in the open-source CLI coding agent category. Star counts and commit velocity shift quickly, so treat this as a dated snapshot rather than a durable ranking.

Architecture: Aider is written in Python and uses the `litellm` library as its LLM abstrac-

tion layer, which means it can talk to virtually any LLM provider that litellm supports. Its edit format is model-dependent: `diff` uses search/replace blocks, while `udiff` uses a unified-diff-derived format; `whole` rewrites the entire file. Aider chooses a suitable format for the model unless you override it. Accepted changes can then be committed automatically. Aider supports several edit modes:

- **diff** — search/replace blocks; the common default for supported models
- **udiff** — unified-diff-derived edit blocks
- **whole** — the model rewrites the entire file (less efficient, sometimes more reliable for small files)
- **architect mode** — a two-step process where one model plans the changes and a second cheaper model applies them, reducing cost

Installation:

```
pip install aider-chat
# or with uv
uv tool install aider-chat
```

LLM provider support: Aider supports all major providers via litellm, including:

- Anthropic (Claude family — `claude-sonnet-5`, `claude-opus-5`, `claude-haiku-4-5-20251001`)
- OpenAI (current GPT-5.6 and Codex model aliases exposed by the installed Aider/litellm release)
- Google (`gemini-3.1-pro-preview` via Vertex AI or AI Studio)
- AWS Bedrock
- Azure OpenAI
- OpenRouter (verified — full setup documented at `aider.chat/docs/llms/openrouter.html`)
- Ollama for local models
- LM Studio
- Groq, Mistral, Cohere, and many others

OpenRouter setup (verified):

```
export OPENROUTER_API_KEY=sk-or-...
aider --model openrouter/anthropic/claude-sonnet-5
# or any OpenRouter-hosted model:
aider --model openrouter/deepseek/deepseek-v4-pro
aider --model openrouter/google/gemini-3.1-pro-preview
```

Local LLM setup (Ollama):

```
# Start Ollama with a coding model first (example tag; verify the current model catalogue)
ollama pull qwen2.5-coder:7b

# Then run Aider
aider --model ollama/qwen2.5-coder:7b
```

Agentic capabilities: Aider can run shell commands via the `/run` command and in `--auto-run` mode will execute suggested commands automatically. It maintains a context

of added files and can be instructed to add more mid-session. It supports `/web` for fetching URLs into context and can integrate with test runners. The workflow is: add files to context → give instruction → model proposes diffs → Aider applies and commits → repeat.

Git integration: Every accepted change is automatically committed. This is tightly integrated — you can see the full history of AI changes in `git log`. There is also an `--auto-commits` flag to control this behaviour.

Shell execution: Via `/run <command>` or `--auto-run`. Aider can also be configured with `--test-cmd` to run tests automatically after each change.

MCP support: Still absent as of late July 2026. RFC #4506 proposing native MCP support remains open, and an exploratory PR (#3937) was closed without shipping. This remains a meaningful gap compared to Claude Code and other MCP-native tools.

Extensibility: Aider has limited plugin architecture compared to Claude Code. Configuration is via `.aider.conf.yml` and environment variables. There is no equivalent to Claude Code's skills or hooks system. What it lacks in extensibility it compensates for in simplicity and reliability.

Modes:

- **code** — the default mode, edits files
- **architect** — planning model + editing model split
- **ask** — ask questions about code without editing
- **help** — help with Aider itself

Cost model: Open source, free to use. You pay only for the LLM API calls you make. The cost of a session depends on the selected model, context, and number of turns; use the provider's current rate card rather than a fixed per-session estimate. With OpenRouter you can route to cheaper models.

Strengths: - Most CLI-native and architecturally similar to Claude Code of all open-source options - Excellent git integration — every change is tracked - Wide provider support via litellm - OpenRouter support is well-documented and works reliably - Mature, battle-tested codebase with high benchmark performance - Supports local LLMs via Ollama - Very low overhead — just Python and an API key

Weaknesses: - No MCP support, and no committed timeline for one (RFC open, prior PR abandoned) - No skills/hooks system — less extensible than Claude Code - Context management is manual (you explicitly `/add` files) - Less capable at long multi-step autonomous tasks compared to Claude Code - No built-in web search or browser use - Development cadence

has slowed relative to faster-moving competitors

5.2 OpenCode

What it is: OpenCode is an open-source coding agent with a rich terminal UI (TUI), a client/server architecture, and support for a very wide range of LLM providers.

Author and licence: Originally built by the SST team; the project rebranded in early 2026 under a new organisation, Anomaly Inc., and the repository moved from `sst/opencode` to `anomalyco/opencode`. Open source.

Community size and activity: OpenCode has a large, fast-moving open-source community and a broad provider ecosystem. Exact star counts, release numbers, and provider totals change quickly; use the current repository and provider documentation rather than a hard-coded snapshot.

Architecture: Current OpenCode is primarily a TypeScript/Bun application with a client/server architecture and a full terminal user interface (TUI). The repository also contains a small Rust component, but describing the product as a Go/Bubble Tea binary is obsolete. It integrates with the AI SDK and Models.dev for LLM provider abstraction. It also supports Language Server Protocol (LSP) for code intelligence, meaning it can provide type-aware, semantically accurate code context rather than just raw file contents. Multi-session support allows you to maintain separate contexts for different tasks.

Installation:

```
# via npm (cross-platform)
npm install -g opencode-ai

# or directly via binary release
curl -fsSL https://opencode.ai/install | bash
```

LLM provider support: OpenCode supports 75+ LLM providers (verified) including:

- OpenAI (ChatGPT Plus/Pro can be authenticated from the provider picker)
- Google (Gemini via AI Studio or Vertex)
- AWS Bedrock
- Azure OpenAI
- OpenRouter
- Ollama (local models)
- LM Studio
- Grok (xAI)
- And many more via the AI SDK / Models.dev integration

This is the broadest provider flexibility of any CLI tool in the category.

Important change — subscription access: OpenCode's providers documentation says that previous Claude Pro/Max plugins are no longer bundled as of v1.3.0 and are explicitly prohibited by Anthropic. The same documentation lists ChatGPT Plus/Pro, GitHub Copilot, and GitLab Duo as zero-setup subscription providers. For Claude, use an Anthropic API key unless you have verified the current authorisation path and its terms; do not treat the old bundled-plugin route as supported.

OpenRouter and Ollama: Both confirmed as supported providers. Configuration is via `~/.config/opencode/opencode.json` or a project-level `opencode.json`.

Agentic capabilities: OpenCode can read and edit files, run shell commands, and iterate autonomously. The TUI provides a chat interface that feels more like a dedicated application than a REPL. LSP integration means it can navigate symbol definitions, find references, and understand project structure at the type level.

MCP support: OpenCode supports MCP servers. This is a significant advantage over Aider for users who have invested in an MCP ecosystem.

Cost model: Open source, free to use. Your cost depends on the selected provider: API billing, an eligible subscription, or an optional OpenCode Zen/Go plan.

Strengths: - Broadest LLM provider support of any CLI tool (75+ endpoints verified) - Rich TUI with a more polished interactive experience than most CLI tools - LSP integration for semantic code understanding - MCP support - Extremely active development, now with a large mainstream user base - Client/server design supports multiple front ends, including the TUI and a beta desktop application - Native ChatGPT subscription support

Weaknesses: - Relatively new compared to Aider — less battle-tested - TUI approach is slightly less scriptable than a pure REPL like Aider - Claude subscription access needs particular care: the old bundled plugin route is prohibited by Anthropic and is no longer shipped - Org/rebrand churn (`sst/opencode` → `anomalyco/opencode`) means older documentation and links may be stale

5.3 Goose (by Block)

What it is: Goose is an open-source, on-device AI coding agent developed by Block (formerly Square, the company associated with Jack Dorsey). It is available as a CLI, a desktop application, and an API. Its primary differentiator is an extension system built natively on MCP.

Author and licence: Developed by Block, Inc. and now maintained in the `aaif-goose/goose`

project. Apache 2.0 licence. The current repository is `github.com/aaif-goose/goose`; older links to `block/goose` are stale.

Architecture: Goose is a native Rust agent available as a desktop app, CLI, and API. Agent execution runs on your machine; model calls go wherever you configure them, including local Ollama. Its extension system is built directly on MCP rather than a Goose-proprietary format — MCP is a core architectural pillar of the tool, not a bolted-on integration. Goose documents 70+ extensions and broad compatibility with the MCP ecosystem, so existing MCP investments generally carry over directly.

Installation:

```
# macOS, Linux, or Windows
curl -fsSL https://github.com/aaif-goose/goose/releases/download/stable/download_cli.sh | bash
```

LLM provider support: Goose is model-agnostic and supports multiple provider configurations simultaneously, with 15+ documented provider integrations including Anthropic, OpenAI, Google, Groq, and Ollama. It does not lock you to a single provider and allows per-task model configuration.

Local LLM support: Confirmed support for Ollama — this is a core feature of the on-device design, not a partial or unverified claim.

Agentic capabilities: Goose can edit files, run shell commands, use its extension system to call external tools, and iterate autonomously on tasks. It has a particularly strong story for DevOps and infrastructure tasks.

MCP support: Confirmed and central to the architecture, not a hedged or disputed claim. Goose's extension system is built on MCP and documents 70+ extensions. Goose is part of the Linux Foundation's Agentic AI Foundation; that governance relationship is separate from the practical question of whether a given MCP server's transport and authentication work in Goose.

Extensions: The extension system is the key differentiator, and it is MCP-native: extensions are effectively MCP servers, giving Goose access to the same growing ecosystem Claude Code and other MCP-compatible tools draw on.

Strengths: - Strong corporate backing from Block — less likely to be abandoned - On-device execution philosophy — good for privacy - MCP-native extension system with 70+ documented extensions and broad MCP ecosystem compatibility - Desktop app available for non-terminal users - Apache 2.0 licence - Model flexibility across 15+ providers - Part of the Linux Foundation's Agentic AI Foundation ecosystem

Weaknesses: - Less pure CLI tool than Aider — the desktop app orientation means some

rough edges in headless terminal use - Younger MCP ecosystem integration story than Claude Code's, despite the architecture being sound

5.4 Gemini CLI (by Google)

What it is: Gemini CLI is Google's open-source terminal AI agent, released in mid-2025. It is powered by the Gemini model family. Google announced a transition to Antigravity CLI: from 18 June 2026, Gemini CLI stopped serving individual Google AI Pro, Google AI Ultra, and free Gemini Code Assist accounts, while enterprise Gemini Code Assist and API/Vertex paths remain supported.

Author and licence: Google Cloud/Developer Products (not Google DeepMind, contrary to earlier reporting). Apache 2.0 licence. Open source at `github.com/google-gemini/gemini-cli`.

Community size: Gemini CLI saw extremely rapid adoption after launch — it became one of the fastest-growing GitHub repositories in history within weeks of release.

Architecture: Gemini CLI is a Node.js-based CLI agent. It uses Gemini's Pro-tier model by default, which provides a very large context window — among the largest of any CLI coding tool. This means you can load entire codebases into context rather than managing file additions manually.

Installation:

```
npm install -g @google/gemini-cli
gemini
```

LLM provider support: Gemini CLI is primarily designed for Google's Gemini models; exact aliases exposed by the installed release change as Google migrates the consumer path towards Antigravity CLI. The current Gemini API catalogue includes Gemini 3.8 Flash, 3.7 Flash, 3.5 Flash-Lite, and 3.1 Pro Preview. It is not model-agnostic in the same way as Aider or OpenCode.

Access and quotas: Google's transition announcement says individual free, Pro, and Ultra access moved to Antigravity CLI on 18 June 2026. Enterprise Gemini Code Assist licences remain supported, and Gemini CLI remains available through API-key and Vertex AI authentication. A Gemini CLI plans page still displays a legacy-looking individual free tier, so treat that page as inconsistent with the transition announcement and test the actual sign-in path before relying on Gemini CLI for individual use.

MCP support: Gemini CLI supports MCP servers, making it one of the CLI tools with confirmed MCP integration alongside Claude Code and Goose. Configuration is via a `~/.gemini/settings.json` file.

Extensions: Gemini CLI has an extensions mechanism that allows adding capabilities beyond the default file editing and shell execution.

Agentic capabilities: Gemini CLI can read and edit files, run shell commands, search the web, and iterate autonomously. Its large context window means it is uniquely capable at tasks involving entire-codebase understanding.

Cost model: Individual Google-account access has moved to Antigravity CLI. Gemini CLI remains usable for enterprise Gemini Code Assist accounts and through Gemini API/Vertex AI authentication; those paths have their own quotas, billing, and data-use terms.

Strengths: - Large context window — can load entire codebases - MCP support - Open source (Apache 2.0) - Google backing means long-term support likely - Very active development

Weaknesses: - Locked to Gemini models — no OpenRouter or third-party model support - Individual consumer access is no longer the supported Gemini CLI path; migrate to Antigravity CLI - Gemini models, while capable, may not match Claude's code quality for your specific use cases - Less battle-tested than Aider - Node.js dependency

5.5 Plandex

What it is: Plandex is an open-source CLI agent with a distinctive planning-first approach. Rather than immediately executing changes, it plans multi-file, multi-step tasks and builds up pending changes for user review before applying.

Author and licence: Created by Dane Schneider (@danenania). MIT licence. Open source at `github.com/plandex-ai/plandex`. Roughly mid-teens-thousands GitHub stars as of late July 2026, currently at v2.2.1.

Architecture: Plandex is written in Go and uses a client-server architecture. The server can be run locally or self-hosted on a cloud provider. Changes are accumulated in a "pending changes" buffer that you can review, modify, or reject before applying to the working tree. This makes Plandex the most review-oriented tool in the category — it assumes you want to understand what's happening before it happens.

Installation:

```
# Install client
curl -sL https://plandex.ai/install.sh | bash

# Use cloud server (free tier available)
plandex sign-in

# Or self-host the server
docker-compose up -d  # with the plandex-server repo
```

LLM provider support: Plandex supports OpenAI models directly and OpenAI-compatible APIs, which includes many providers. OpenRouter support via the OpenAI-compatible endpoint is possible.

Self-hosting: A key differentiator — you can run the entire Plandex server yourself, meaning no data leaves your infrastructure.

Agentic capabilities: Plandex is specifically designed for large, multi-step tasks: "implement this feature across 20 files." Its planning mechanism handles long sequences of related changes better than tools that tackle each change independently.

Strengths: - Planning-first approach is excellent for large refactors - Self-hosting option for data sovereignty - Review buffer means less risk of unexpected changes - Good at long-horizon multi-file tasks

Weaknesses: - Planning overhead makes it slower for quick edits - Less interactive than Aider or OpenCode - Client-server architecture adds complexity - LLM provider support less broad than Aider/OpenCode - Smaller community

5.6 SWE-agent

What it is: SWE-agent is a research tool from Princeton NLP lab, designed primarily for automated software engineering on GitHub issues. It achieves high scores on the SWE-bench benchmark. It is less a daily-driver tool and more a demonstration of what autonomous agents can accomplish on discrete bug-fix tasks.

Author and licence: Princeton NLP group. MIT licence. `github.com/SWE-agent/SWE-agent`.

Architecture: SWE-agent uses a Docker-based sandboxed environment. The Agent-Computer Interface (ACI) pattern it introduces provides structured tools for file navigation, editing, and execution in a way that is particularly suited for benchmark tasks. Its strongest benchmark claims in this document are date-stamped to early 2026 and should be treated as historical snapshots, not permanent rankings. Active development in this lineage has shifted to a sibling project: **mini-swe-agent**, a ~100-line Python reimplementation of the same core idea. If you're evaluating this family today, check both projects and verify current SWE-bench standings before making a decision.

Intended use: Automated bug-fixing on GitHub issues, especially in batch/CI contexts. Not designed for interactive development sessions.

Strengths: - Extremely high benchmark performance - Docker sandboxing for safe execution - Good for automated CI/CD pipelines

Weaknesses: - Not a daily driver — requires Docker, not interactive - Research-oriented design means rough UX edges - Slower than interactive tools - Not designed for the ad-hoc, exploratory coding sessions that Claude Code excels at - Primary development momentum has moved to mini-swe-agent; SWE-agent proper sees less active iteration

5.7 OpenAI Codex CLI

What it is: Codex CLI is OpenAI's open-source terminal coding agent, open-sourced 16 April 2025. It is OpenAI's direct answer to Claude Code and Gemini CLI: a terminal-native agent that reads and edits files, runs shell commands, and iterates autonomously, with tight integration into ChatGPT's subscription plans.

Author and licence: OpenAI. Open source at `github.com/openai/codex`. Roughly high-five-figures to low-six-figures GitHub stars (late-July 2026 snapshot); the codebase is 96.5% Rust.

Architecture: Codex CLI is a Rust binary, giving it a fast startup and low overhead compared to Node.js- or Python-based competitors. It defaults to OpenAI's own hosted models but also supports local model backends.

Installation:

```
npm install -g @openai/codex
# or via the standalone Rust binary release
codex
```

Model support: The current Codex subscription lineup is the GPT-5.6 family: Sol, Terra, and Luna. Sol is the high-capability choice, Terra balances capability and allowance, and Luna stretches usage furthest. Available models depend on plan and may change, so `/model` and `/status` are more reliable than a hard-coded model name in a long-lived guide.

The GPT-6 Astra model is also available in the current OpenAI model catalogue; use the installed Codex `/model` listing rather than assuming every plan exposes every model.

Local model support: Codex CLI can run against local models via an `--oss` flag, with documented Ollama and LM Studio providers. This gives it a genuine offline/local-only mode, something Gemini CLI lacks entirely.

Bundled pricing: Codex is included in ChatGPT Free, Go, Plus, Pro, Business, and Enterprise plans. Allowances vary by plan and model; the official pricing page publishes five-hour local-message limits and notes that additional weekly limits may apply. Users who exhaust an allowance can buy credits on eligible plans or run additional local tasks with an API key at standard API rates. This mirrors Claude Code's subscription bundling more closely than a pay-per-token CLI tool like Aider.

Configuration: Config lives at `~/.codex/config.toml`, covering model selection, sandboxing behaviour, and approval policy for shell commands.

Agentic capabilities: Codex CLI can read and edit files, run shell commands with configurable approval levels, and iterate autonomously across multi-step tasks, broadly comparable in scope to Claude Code and Gemini CLI.

Strengths: - Current GPT-5.6 Sol/Terra/Luna choices let you trade capability against how quickly you consume the plan allowance - Fast, low-overhead Rust binary - Genuine local-model mode via `--oss` (Ollama and LM Studio) — not just a cloud-only tool - Bundled into existing ChatGPT subscriptions, so many users already have access with no incremental cost - Very large and fast-growing community (high-five-figure/low-six-figure star range)

Weaknesses: - Best experience is tied to OpenAI's own models and ChatGPT plans; using it as a purely model-agnostic tool is a secondary use case - Newer than Aider/OpenCode as a CLI-first product, despite OpenAI's scale - Model availability and usage allowances change quickly, making static comparisons easy to age

5.8 Amp (Sourcegraph)

What it is: Amp is Sourcegraph's hybrid CLI-plus-cloud coding agent. It pairs a local CLI with cloud-hosted "Orbs" for offloading longer-running or parallel work. Orbs are remote machines where agents can run without supervision; they pause after inactivity and are billed by the minute.

Author and licence: Sourcegraph.

Architecture: Amp combines a local CLI with cloud-side Orbs for longer-running or parallel work. Sourcegraph heritage informs the product, but this survey does not assume that every Sourcegraph code-intelligence feature is available inside Amp. The CLI handles local interactive work; Orbs handle cloud-side execution.

Deployment boundary: Enterprise customers can now run self-hosted Orbs in their own AWS, Google Cloud, or Azure account and network. That keeps the repository checkout, build output, and paused Orb files in the customer's cloud account, but it does not make Amp self-contained: the Amp service still runs the conversation, and file contents, command output, and code changes may travel through Amp and the selected model provider.

LLM provider support: Multi-model, with supported provider-key configuration and plugin/skill/hook extensibility. It is flexible, but "model-agnostic" overstates the default curated product surface.

Cost model: Amp has a free Hobby tier (pay-as-you-go orbs, no token fees or limits), a $20/month Individual tier (45,000 orb minutes, unlimited repos), a Teams tier (pooled credits, no extra platform charge on top of members' own plans), and a custom-priced Enterprise tier (pooled credits only, plus SCIM/audit logs/IP allowlisting) (as of 2026-09-12). Individual and non-enterprise workspaces do not pay a model markup; Orb compute is metered separately by the minute with automatic pause for idle instances. Check the live pricing page before budgeting: Amp's model and compute rates are usage-based and have changed structure before.

Strengths: - Sourcegraph-backed product lineage and cloud execution for longer tasks - Hybrid local/cloud model gives a path to offloading long-running or parallel tasks without leaving the CLI workflow - Supported multi-model routing with extensibility via plugins, skills, and hooks - No markup on individual pay-as-you-go pricing

Weaknesses: - Orb compute and agent inference are separate meters, so a subscription is not an all-in fixed-cost plan - Newer product identity means less accumulated community track record than Sourcegraph's older Cody product - Cloud "Orbs" component means it is not a purely local/offline tool in the way Aider or Goose can be; self-hosted Orbs still depend on Amp's hosted control plane

5.9 Deprecated and Unverified Tools

5.9.1 Mentat

What it is: Mentat was a Python-based CLI coding tool and an early entrant in the space, taking a conversational approach to code editing with explicit context management.

Status: Fully archived and read-only since 7 January 2025 — this is a firm end-of-life, not a gradual slowdown. Note for anyone searching for current status: "Mentat" is now also used by an unrelated GitHub bot product from Abante AI, which can make search results confusing — the archived coding CLI and the Abante AI bot are different products that happen to share a name.

Note: For new users, Aider is a better choice in the same category — more active, more features, larger community.

Roo Code (fork of Cline) — also deprecated: Roo Code, a fork of Cline created in 2024 to move faster on multi-agent workflows (its "Boomerang Tasks" feature) and broader model support, was archived on 15 May 2026 and is now read-only. The project has publicly directed users to alternatives such as Cline and ZooCode. There is no live "current tool" entry for Roo Code in this document — by the time of this refresh it is itself historical.

5.10 Qwen Code and Kimi CLI

What they are: These are CLI coding agents from Chinese AI labs — Qwen Code from Alibaba's Qwen team and Kimi CLI from Moonshot AI. Both are classified alongside Claude Code and Aider as CLI tools by Artificial Analysis's taxonomy. Both are positioned as competitive alternatives particularly for users who want model diversity or prefer models strong at certain programming languages.

Provider flexibility: Qwen Code uses Qwen models (strong at code, particularly multilingual/non-English codebases; current coding-focused models include `qwen3-coder-next`, `qwen3-coder-plus`, and `qwen3-coder-480b`). Kimi CLI uses Kimi's models with a very long context window.

Note: These tools were not covered by this round's research refresh. Treat specific capability claims with appropriate scepticism until you test them directly.

6 IDE Extensions and Hybrid Tools

6.1 Cline (formerly Claude Dev)

What it is: Cline is an open-source autonomous coding agent that operates primarily as a VS Code extension. It is model-agnostic, OpenRouter-compatible, and one of the most capable agentic tools available for IDE-based workflows. It has since expanded well beyond VS Code and beyond the IDE entirely, moving it towards hybrid territory.

Author and licence: Open source at `github.com/cline/cline`. MIT licence.

Community size: Roughly low-60k GitHub stars (late-July 2026 snapshot). This makes it one of the most popular tools in the entire AI coding agent space, not just this subcategory.

Platform reach: Beyond VS Code, Cline runs in JetBrains IDEs and can connect to ACP-compatible editors such as Zed and Neovim through its CLI. It is now a real terminal agent as well as an IDE extension; the CLI requires Node.js 20+ and supports interactive, headless, and JSON-output workflows.

Architecture: Cline runs as a VS Code extension and has deep access to the VS Code API — language servers, the file system, the integrated terminal, and the browser (via Puppeteer integration). It operates in two modes:

- **Plan mode** — Cline analyses the task and produces a plan before executing
- **Act mode** — Cline executes changes directly

This Plan/Act split allows you to review the strategy before any files are modified.

LLM provider support: Cline is model-agnostic. It supports:

- Anthropic (Claude family)
- OpenAI (GPT-4o, o1)
- Google (Gemini)
- AWS Bedrock
- Vertex AI
- OpenRouter (verified — listed on OpenRouter's works-with-openrouter page)
- Ollama for local models
- LM Studio
- Any OpenAI-compatible API endpoint

OpenRouter setup:

```
In VS Code: Open Cline extension settings
API Provider: OpenRouter
API Key: sk-or-...
Model: anthropic/claude-sonnet-5 (or any OpenRouter model)
```

Agentic capabilities: Cline can:

- Read and edit any file in the workspace
- Execute shell commands in the integrated terminal
- Create and delete files/directories
- Use the browser (view pages, click, type, screenshot)
- Run searches across the codebase
- Read error output and iterate automatically

Browser use is a standout capability — Cline can actually open a browser, navigate to a URL, interact with a web page, and read the results. This enables tasks like: "go to this API docs page and implement the integration."

MCP support: Cline supports MCP servers. Configuration is via the Cline settings JSON. This means your existing MCP server investments (filesystem servers, database servers, custom tools) are portable to Cline.

Extensibility: Beyond MCP, current Cline has project rules and skills under `.cline/`, lifecycle hooks, plugins, and a system-prompt override. Older `.clinerules` guidance may still appear in documentation, but `.cline/rules/`, `.cline/skills/`, `.cline/hooks/`, `.cline/plugins/`, and `.cline/mcp.json` are the current CLI configuration surfaces.

CLI mode: The standalone CLI makes Cline usable outside any IDE. It supports plan mode, provider/model selection, MCP management, plugins, hooks, schedules, a background

hub, ACP integration, and structured JSON output. Windows support remains a separate question in the current installation documentation, so check the platform requirements before standardising on it.

Cost model: Open source. You can use Cline's hosted provider, a direct provider/API key, or local Ollama/LM Studio; price and quota depend on the selected path. OpenRouter usage is billed by the underlying model.

Team note: Cline remains independent, open source, and actively developed as of end-July 2026.

Roo Code lineage: Roo Code forked from Cline in 2024 to push further on multi-agent workflows (Boomerang Tasks) and broader model support. It was archived on 15 May 2026 and is now read-only; most users evaluating that lineage now compare Cline and ZooCode.

Strengths: - Extremely capable agentic tool — one of the best - Model-agnostic with OpenRouter support verified - Browser use capability is notable among open-source tools - MCP support - Plan/Act mode for controlled execution - Very large and active community - Now reaches far beyond VS Code (JetBrains, ACP-compatible editors, and CLI)

Weaknesses: - The CLI and extension expose different workflows, and platform support is not uniform - Browser use capability means it can have unintended side effects if not supervised - Less scriptable than pure CLI tools

6.2 Continue.dev

What it is: Continue.dev is an open-source AI coding assistant for VS Code and JetBrains IDEs. It combines autocomplete, chat, and agent capabilities in a single extension, with deep configurability via a YAML config file. Since September 2025 it also ships a headless CLI, so it is no longer purely IDE-integrated.

Author and licence: Open source at `github.com/continuedev/continue`. Apache 2.0 licence.

Architecture: Continue.dev is structured around three core features:

1. **Autocomplete** — inline code completions as you type
2. **Chat** — conversational interface about code, with codebase context
3. **Agent** — agentic mode that edits files based on instructions

Configuration lives in `~/.continue/config.yaml` and can specify different models for different features (e.g., a fast local model for autocomplete, a more capable cloud model for agent tasks).

CLI mode (`cn`): Continue.dev shipped a headless `cn` CLI in September 2025, offering both a TUI interactive mode and a scriptable/background mode (`cn -p "prompt"` for one-shot invocation, plus async background jobs). This is a supplementary capability, not a pivot away from the IDE extension — Continue.dev is still primarily used as an IDE assistant, with the CLI extending it into scripts, CI, and terminal-first workflows.

LLM provider support: Continue.dev has one of the broadest provider lists:

- Anthropic, OpenAI, Google
- OpenRouter
- Ollama (confirmed at localhost:11434 by default, remote connections via `apiBase`)
- LM Studio, llama.cpp
- Groq, Together AI, Replicate
- Self-hosted models via any OpenAI-compatible endpoint

Ollama setup (verified):

```yaml
# ~/.continue/config.yaml
models:
  - title: "Local coding model"
    provider: ollama
    model: qwen2.5-coder:7b  # verify the current Ollama model tag
    # For remote Ollama:
    # apiBase: http://192.168.1.100:11434
```

Agent mode: Continue.dev's agent mode allows it to make multi-file edits based on instructions. Tool support depends on the underlying model — models must support function/tool calling. Note: some models that claim tool support may not work reliably in agent mode (this was partially corroborated in the research, though the specific claim was refuted as too broad).

Extensibility: Beyond MCP, Continue.dev supports a block system where community-built extensions ("blocks") add new context providers, tools, and model configurations. This is effectively a plugin marketplace for the IDE assistant workflow.

Strengths: - Best-in-class for IDE users who want full local LLM support - Works in both VS Code and JetBrains - Highly configurable via config.yaml - Autocomplete + chat + agent in one package - OpenRouter and Ollama support confirmed - Headless `cn` CLI extends it into scripts, CI, and terminal workflows - Open source, Apache 2.0

Weaknesses: - Still primarily IDE-integrated; the CLI is supplementary, not a terminal-first redesign - Agent mode quality depends heavily on the model's tool-calling capabilities - Less capable at long agentic chains than Cline or Claude Code - JetBrains support is less mature than VS Code

7 Dedicated AI IDEs

7.1 Cursor

What it is: Cursor is a VS Code fork with deep AI integration baked into every layer of the editor. It has become one of the most widely adopted commercial AI coding tools, with a large user base and reported ARR in the hundreds of millions.

Company: Anysphere, Inc. Private company, significant VC backing.

Pricing:

- **Hobby** — Free — limited completions and requests per month
- **Pro** — $20/month — usage-based credit pool (not a fixed request count)
- **Pro+** — $60/month — larger credit pool for heavier usage
- **Ultra** — $200/month — highest credit pool, priority access to frontier models
- **Teams** — $40/user/month standard or $120/user/month Premium — team features, admin controls, and expanded usage/features
- **Enterprise** — custom

Billing moved from a fixed "500 fast requests/month" allowance to a usage-based credit pool in mid-2025 — treat any "N requests/month" framing of Cursor pricing as obsolete, not merely dated.

Architecture: Cursor extends VS Code with AI capabilities at multiple layers: inline completions, a chat sidebar, and Agent mode. It maintains a shadow workspace where it can test proposed changes before applying them. Current Cursor supports multiple parallel agents and cloud/background workflows; exact concurrency and branch limits are product- and plan-dependent, so avoid treating a fixed agent count as a durable limit.

LLM models available: Cursor Pro gives access to current-generation Claude, GPT, and Gemini models, alongside Cursor's own in-house model, Composer. Cursor model/version details change frequently; check Cursor release notes for the exact current model lineup.

Terminology note: "Composer" and "Agent mode" are two different things, not interchangeable. Composer is the name of Cursor's own in-house model — Anysphere trains and ships it as one of the model options in the picker. Agent mode is the autonomous multi-file execution feature: it accepts a task, proposes a plan, and executes changes across multiple files, and can run terminal commands in Yolo mode (automatic execution without prompting). Agent mode can be pointed at Composer or at any of the other bundled models.

Context mechanisms:

- `@codebase` — full codebase semantic search
- `@docs` — pull in documentation from URLs
- `@web` — web search integration
- `@file`, `@folder` — explicit file/folder references
- `@git` — reference git history and diffs

Rules: Current Cursor rules live primarily in `.cursor/rules/*.mdc` and global rules. `.cursorrules` is a legacy format retained for migration, not the preferred format for new projects.

Current Cursor pricing documentation also lists MCPs, skills, and hooks on its paid plans. Treat plan eligibility and implementation details as volatile, but do not describe Cursor as having only rules-file customisation.

Provider flexibility: Limited. While Cursor allows "BYO API key" (bring your own Anthropic/OpenAI key), it is not natively designed for OpenRouter or arbitrary providers, and it has no local LLM support. The product is optimised for the models it bundles.

Strengths: - Polished, deeply integrated AI coding experience - Agent mode is very capable, and now supports parallel multi-branch agents - Semantic codebase indexing enables large-codebase context - Large community, extensive documentation, many tutorials - Tab completion is best-in-class

Weaknesses: - IDE-bound — not a CLI tool - Limited model flexibility — still not OpenRouter compatible, no local model support - Proprietary — you are dependent on Anysphere remaining a going concern and maintaining pricing - Heavier than a terminal tool — requires a full VS Code instance - Usage-based credit pricing makes monthly cost less predictable than the old flat-request model

7.2 Windsurf (now Devin Desktop, Cognition AI)

What it is: Windsurf was an AI-first IDE built by Codeium, positioned as a direct competitor to Cursor. Its standout feature was the Cascade agent, which used "Flow" — a system that maintained contextual awareness across an entire coding session rather than treating each interaction as independent. The product has since been acquired and rebranded; treat "Windsurf" as the legacy name and "Devin Desktop" as the current one.

Ownership timeline: Cognition announced its acquisition of Windsurf on **14 July 2025**; the transaction amount is not established by the current first-party sources used here. The product was subsequently presented as **Devin Desktop**. Treat Windsurf as the legacy name

and Devin Desktop as the current product name, but verify the current migration and feature terminology before relying on old Cascade documentation.

Company: Cognition AI.

Pricing: Devin Desktop has its own pricing ladder, separate from Cognition's cloud Devin product (see the Devin entry under Cloud Platform Agents — do not conflate the two):

- **Free** — limited quota
- **Pro** — $20/month
- **Max** — $200/month
- **Teams** — $80/month base + $40/seat

Architecture: Devin Desktop remains a VS Code-derived desktop IDE. Its current product page describes agents running on the local machine and integrations such as extensions, plugins, and MCP; that does not establish local LLM inference.

LLM models: Devin Desktop uses Cognition-managed inference by default, with some degree of model selection in higher tiers.

Provider flexibility: Cognition-managed inference is the default; the current product page does not establish a general Ollama, LM Studio, or OpenRouter path. Treat local execution of an agent as distinct from running the model locally.

Strengths: - Local-machine agent execution and a polished IDE experience - Historically cheaper tiers than Cursor at the low end - Autocomplete remains strong (inherited from Codeium's original product) - Clean, polished IDE experience

Weaknesses: - IDE-bound - Two ownership changes and a rebrand in under a year — evaluate stability before committing long-term - Less model flexibility than Cursor - Smaller community than Cursor; documentation is still catching up to the rebrand - Easy to confuse with Cognition's separate cloud Devin product, which has different pricing and a different execution model

7.3 Zed AI

What it is: Zed is a high-performance code editor written in Rust, designed for speed and minimal latency. Its AI integration (Zed AI) brings Claude as the default model into this fast editor experience.

Architecture: Zed is not a VS Code fork — it is a ground-up implementation in Rust with native GPU acceleration. This makes it significantly faster than Electron-based editors (VS Code, Cursor, Devin Desktop). AI features are integrated via an assistant panel. Zed's

Agent Client Protocol (ACP, launched January 2026) is a distinct interoperability feature none of the other dedicated AI IDEs offer: it lets Claude Code, Codex CLI, Gemini CLI, and OpenCode run directly inside Zed as pluggable agents, rather than requiring Zed's own agent to do everything.

LLM models: Claude is the default. The supported provider list is now extensive: OpenAI, Gemini, Amazon Bedrock, DeepSeek, GitHub Copilot, LM Studio, Mistral, Ollama, OpenRouter, and Vercel.

Pricing: Zed the editor is free and open source. Zed AI is billed separately on token-based usage, restructured into three tiers:

- **Personal** — free permanently — 2,000 accepted edit predictions/month
- **Pro** — $10/month — unlimited edit predictions, $5 of tokens included, overage billed at API list price + 10%
- **Business** — $30/seat/month

Strengths: - Extraordinarily fast editor — the best performance of any AI-integrated editor - Open source editor core - Genuinely broad provider flexibility, including local (Ollama) and OpenRouter — no longer just "supports configuring alternative backends," both are confirmed Yes - ACP lets you bring your preferred CLI agent (Claude Code, Codex CLI, etc.) into the editor instead of relying solely on Zed's own agent

Weaknesses: - Smaller plugin/extension ecosystem (incompatible with VS Code extensions) - Native agentic mode is still less mature than Cursor's or Devin Desktop's - Windows is supported alongside macOS and Linux - Token-based overage billing can be less predictable than a flat monthly fee for heavy users

7.4 Google Antigravity

What it is: Google Antigravity is Google's agent-first development platform. It includes Antigravity CLI, the Antigravity 2.0 desktop application, IDE integrations, an SDK, and managed agents in the Gemini API. It is the announced migration target for individual Gemini CLI users.

Company: Google (Cloud/Developer Products, the same organisation behind Gemini CLI — not Google DeepMind).

Pricing and access: Antigravity has a $0 individual tier with basic weekly rate limits, Google AI Pro with higher limits and a flexible AI-credit pool, and Google AI Ultra with the highest quotas and access to third-party models. Google Cloud customers can use Anti-

gravity through the Gemini Enterprise Agent Platform with consumption-based API pricing. Antigravity's plans page says there is no BYOK or custom endpoint for additional account quota, although the CLI can separately use a Gemini API key for direct Gemini-compatible inference.

LLM models: The current Antigravity product pages list Gemini models, Claude Sonnet and Opus 4.6, and `gpt-oss-120b` among the available models; exact availability depends on plan and surface. Managed Antigravity agents are powered by Gemini 3.7 Flash by default and can be configured to use supported Gemini 3.x models.

Why it matters here: Antigravity is now the relevant Google alternative for both terminal-first and desktop workflows. Antigravity CLI is a Go-based terminal tool with skills, hooks, subagents, and plugins; Antigravity 2.0 is a standalone desktop application for synchronous and asynchronous multi-agent work. The products are new and still evolving, so verify the current plan, model, and regional availability before standardising on them.

Strengths: - Direct migration path for users displaced by Gemini CLI's consumer-account transition - Multi-model product surface, including Gemini, selected Claude models, and open-weight models - Under active development by Google

Weaknesses: - Newest entrant in this table — least battle-tested - Product and pricing details remain volatile across surfaces - No general BYOK/OpenRouter path for extending account quotas

7.5 AWS Kiro IDE and CLI

What it is: Kiro is AWS's agentic development environment, available as a VS Code-based desktop IDE, terminal-native CLI, web surface, and mobile companion. These surfaces share one agent harness and the `.kiro/` project configuration. The IDE supports ordinary editor work plus structured Specs; the CLI adds headless and continuous-integration workflows.

Architecture and extensibility: Kiro combines project steering files, custom agents, skills, hooks, MCP servers, permissions, Powers, and checkpoints/rewind. It also discovers `AGENTS.md` files as steering context. Hooks can run shell commands or send prompts to the agent at events such as prompt submission, tool use, file changes, and agent lifecycle transitions. This makes Kiro one of the closest alternatives to Claude Code's skills-plus-hooks model, although the file formats and event semantics differ.

Provider and model support: Kiro uses AWS-managed model routing rather than arbitrary provider endpoints or OpenRouter. Its current model selector includes AWS's Auto route, Anthropic Claude models, OpenAI GPT-5.6 variants, and selected open-weight mod-

els. Free users get a limited monthly credit allocation; paid plans expose progressively larger credit pools and access to premium models.

Pricing (31 August 2026): Kiro Free is $0/month with 50 credits. Pro is $20/month with 1,000 credits; Pro+ is $40/month with 2,000; Pro Max is $100/month with 5,000; and Power is $200/month with 10,000. Paid users can buy add-on credits at $0.04 each. Credits are fractional and model-dependent, so request counts are not a reliable substitute for the credit budget.

MCP: Kiro supports local stdio MCP servers and remote HTTP/SSE servers across its IDE, CLI, and web surfaces, with surface-specific differences. Treat remote-server transport and authentication as part of deployment testing rather than assuming every server works everywhere.

Strengths: - The strongest documented skills/hooks/MCP/steering story among the commercial IDE alternatives - Shared configuration across IDE, CLI, and web - Structured Specs for requirements, design, and task execution - CLI and ACP support, not just an editor extension - Clear credit-based pricing with a free tier

Weaknesses: - AWS-managed model routing; no OpenRouter or general BYO-provider path - Credit consumption varies by model and task, so heavy-use cost is not a flat subscription - Closed source and tied to AWS's product and model availability - Newer than the established IDEs; verify regional model access and feature availability

8 Cloud and Web Platform Agents

8.1 Grok Bot (xAI)

What it is: Grok Bot is xAI's commercial, hosted computer-use agent. xAI launched it in beta on **11 August 2026** as a set of persistent AI teammates rather than as another chat session. A Bot has a name, a job, its own conversation and working context, and can turn a repeatable workflow into a skill or routine. Coding is one workload — xAI describes engineering Bots reproducing bugs and handing fixes to other Bots — but the product is aimed at work that crosses applications, websites, inboxes, documents, and business systems.

Architecture: The desktop and mobile applications are thin clients for chat, review, and approvals. The work runs on a persistent cloud computer with a browser, filesystem, and terminal. Bots can operate applications and websites directly, including services without a clean API or Model Context Protocol (MCP) integration. This makes Grok Bot materially different from an API-only agent: it can use the same graphical interfaces a human uses.

All Bots belonging to one user share that user's cloud computer, including its files, browser sessions, and app logins; separate Bots are therefore not separate security boundaries.

Agentic workflow: You message a Bot like a colleague, give it a task and access to the required tools or files, and let it work asynchronously. Bots can remember stable preferences, run recurring routines, collaborate through handoffs, and return when they need approval. The product is closer to a persistent operations teammate than to a terminal REPL. It is not a drop-in replacement for Claude Code's local repository workflow: its execution environment is hosted, and its strongest differentiator is cross-application computer use rather than terminal-native editing.

Access and cost (9 September 2026 snapshot): Grok Bot launched for eligible SuperGrok and Cursor subscribers and later expanded to SuperGrok, paid Cursor, and Cursor Teams plans; enterprise rollout is handled through the Cursor account team. xAI's August 26 announcement says Bot usage is separate from existing Grok and Cursor usage, while the current team documentation describes plan-specific allowances. Treat access, quotas, and billing as volatile and check the live plan matrix before budgeting. Grok Bot is a product-level service; access to Grok models through the xAI API is a separate integration path.

Security and privacy: Consequential actions can pause for approval, with Auto Review available to inspect computer and tool actions before execution. Passwords, passkeys, two-factor codes, CAPTCHAs, and payment confirmations are handed back to the user through computer takeover rather than entered into ordinary chat. Grok Bot requires cloud data storage and does not support Cursor's Legacy Privacy Mode. Optional execution on the user's local computer is separate from the shared cloud computer and asks for approval by default. Start with read-only tasks, least-privilege connectors, and explicit approval boundaries for sending, publishing, purchasing, deletion, permission changes, and production changes.

Strengths:

- Persistent cloud execution that continues while you are away
- Direct browser and desktop-style interaction across applications, including systems without APIs or MCP servers
- Durable Bots, routines, memory, and Bot-to-Bot handoffs for repeatable operational work
- Approval and credential handoff controls for consequential or sensitive actions

Weaknesses:

- Proprietary, hosted, and dependent on xAI/Cursor product access and policy

- Requires cloud storage; the shared computer means one Bot's files and sessions can be available to the user's other Bots
- Not a local-model, self-hosted, or OpenRouter-oriented workflow
- Beta product with rapidly changing plan eligibility, quotas, and administrative controls

8.2 OpenHands (formerly OpenDevin)

What it is: OpenHands is a highly capable open-source AI software engineer designed to solve complex software engineering tasks autonomously. It is the open-source project that most directly competes with commercial cloud agents like Devin.

Author and licence: MIT licence. `github.com/OpenHands/OpenHands` — the project migrated from the `All-Hands-AI` org to `OpenHands`; the new path is now the canonical one.

Community size: Roughly high-70k to low-80k GitHub stars and 7,000+ commits (late-July 2026 snapshot). One of the largest open-source AI coding projects by community size.

Architecture: OpenHands runs in a Docker container with a sandboxed execution environment. This is the key architectural difference from CLI tools — it doesn't run directly on your machine; it runs in an isolated environment with full access to the shell, filesystem, and browser within that sandbox. This makes it safer for autonomous long-running tasks but adds Docker as a dependency. The project has recently restructured its core: the agent code now lives in separate repos (`software-agent-sdk`, `agent-canvas`), and the primary interface is the browser-based **Agent Canvas** rather than the older "Web UI" branding.

Interfaces:

- **Agent Canvas** — the primary interface, a browser-based UI for interacting with the agent
- **CLI** — a lightweight binary available at `github.com/OpenHands/OpenHands-CLI`
- **API** — programmable access for CI/CD integration

Installation:

```
# Via Docker (recommended; the registry's default tag tracks the current runtime)
docker pull docker.all-hands.dev/all-hands-ai/runtime

# Lightweight CLI binary (for terminal-first use)
pip install openhands-cli
```

LLM provider support: OpenHands supports multiple LLM providers via litellm, including Anthropic, OpenAI, Google, and others.

MCP support: OpenHands supports MCP server configuration across three transport types — SSE, Streamable HTTP, and stdio:

```
{
  "mcpServers": {
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    },
    "notion": {
      "url": "https://mcp.notion.com/mcp"
    }
  }
}
```

For stdio servers, a proxy approach (SuperGateway/FastMCP) is recommended over direct stdio for reliability. Note: the claim that OpenHands "natively discovers MCP tools automatically" remains refuted — MCP is configured explicitly, not auto-discovered.

Strengths: - Very capable at complex, long-horizon tasks — closest open-source equivalent to Devin - Docker sandboxing means it can't accidentally break your host system - Full MCP support across three transport types - Massive community and active development - Agent Canvas is more accessible than CLI tools

Weaknesses: - Requires Docker — heavier than CLI tools - Primarily a cloud/web platform — less suitable for quick interactive sessions - Slower to start up than CLI tools (Docker container spin-up) - Not a terminal-first tool despite having a CLI option

8.3 Devin (by Cognition AI)

What it is: Devin was the first high-profile "AI software engineer" capable of autonomously solving complex GitHub issues end-to-end. It is a commercial cloud product from Cognition AI.

Pricing: Devin now has Free ($0), Pro ($20/month), Max ($200/month), Teams ($80/month minimum plus $40/month per full seat), and custom Enterprise plans. Paid self-serve plans include daily and/or weekly usage allowances; extra usage is purchased as on-demand credits at API pricing. ACU terminology is legacy for self-serve but remains relevant to Enterprise billing. This makes the headline price easier to understand, but heavy usage can still exceed the subscription price.

Devin now sits alongside a sibling product, **Devin Desktop** (formerly Windsurf, rebranded June 2026 after Cognition's acquisition of Windsurf) — see the Windsurf/Devin Desktop entry for that product's separate pricing ladder. The two are distinct: Devin is the cloud autonomous-agent product with plan quotas and on-demand credits; Devin Desktop is an IDE and agent command centre.

Strengths: - Very capable for long-horizon autonomous tasks - Well-integrated with GitHub workflows - No longer prohibitively priced for individuals at light-to-moderate usage levels

Weaknesses: - Usage allowances and on-demand credits still make heavy-use costs less predictable than a simple flat subscription - Fully proprietary — no control over the stack - Not a CLI tool

9 Commercial AI Coding Assistants

9.1 GitHub Copilot with Agent Mode

What it is: GitHub Copilot is Microsoft/GitHub's AI coding assistant. It now spans autocomplete, chat, an in-IDE "Agent Mode," and a separate cloud-based "Copilot Coding Agent." These are two distinct products, not one: **Agent Mode** (VS Code, synchronous, runs locally, GA since early 2026) executes edits directly in your editor session; the **Copilot Coding Agent** (cloud, asynchronous, GA September 2025) takes a GitHub issue assignment and opens a pull request without an open editor session. Both superseded the original "Copilot Workspace" preview, which was sunset 30 May 2025 — that name no longer refers to a current product.

Pricing (restructured with usage-based billing, 1 June 2026):

- **Free** — $0/month; 2,000 completions/month
- **Pro** — $10/month; unlimited completions plus included AI credits; additional consumption varies by model and workload
- **Pro+** — $39/month; unlimited completions + $70/month AI credits; premium model access including Claude Opus
- **Individual Max** — $100/month; larger included AI-credit allowance; additional consumption varies by model and workload
- **Business** — $19/user/month
- **Enterprise** — $39/user/month

LLM provider: Copilot was never as OpenAI-locked as it appeared. The built-in model picker includes Claude, Gemini, Kimi, GPT, and other models, with availability varying by client and plan. GitHub now documents two BYOK paths: local BYOK in VS Code, JetBrains, Xcode, Copilot CLI, the Copilot app, and the SDK; and enterprise-managed custom models for Business/Enterprise. Local BYOK can work without a Copilot subscription and can target local or external providers, while enterprise BYOK is server-managed and remains in preview. BYOK and model availability vary by client and feature; code comple-

tions remain on their separate billing path and are not automatically covered by every BYOK configuration.

OpenRouter support: Possible through a compatible local-BYOK/provider integration, but do not present it as one uniform Copilot feature. Client, plan, and organisation policy determine whether a user can configure it; the enterprise custom-model path is still in public preview.

Strengths: - Deep GitHub integration — issues, PRs, code review - Real provider flexibility now, not just a wide built-in model picker - $10/month remains the most affordable commercial completions tier - Enterprise features and security compliance

Weaknesses: - BYOK coverage depends on the client, feature, plan, and organisation policy; local BYOK and enterprise-managed custom models are different paths - Agent Mode and Copilot Coding Agent are two separate mental models to learn, with different sync/async execution semantics - Less suited for non-GitHub workflows

9.2 Amazon Q Developer (formerly CodeWhisperer) — IDE transition to Kiro

What it is: Amazon Q Developer is AWS's AI coding assistant, with deep integration into the AWS ecosystem (rebranded from CodeWhisperer in 2024). Its agentic mode autonomously implements features, refactors, and proposes multi-step changes in both IDE and CLI — this is no longer a completions-only tool, contrary to earlier characterisations.

CLI agentic mode: Supports Claude Sonnet 4 / 3.7 / 3.5 via `/model`, with file read/write, bash execution, and multi-step autonomy — a genuine general-purpose coding agent, not an AWS-console-only assistant.

Pricing:

- **Free** — 50 agentic requests/month
- **Pro** — $19/user/month; 10,000 inference calls (~1,000 user requests)

Important — deprecation in progress: AWS will discontinue support for Amazon Q Developer IDE plugins and paid subscriptions on 30 April 2027; new Q Developer account and subscription signups closed 15 May 2026. Amazon Q Developer in the AWS console and other first-party AWS experiences is not covered by that IDE sunset. **AWS Kiro is the designated successor for IDE/CLI development** (see above) — do not recommend Q Developer as a new IDE investment without flagging this scope and date.

Strengths: - Best-in-class for AWS-heavy workflows - Security vulnerability scanning, IAM

policy generation - CLI agentic mode is now genuinely general-purpose, not AWS-console-limited

Weaknesses: - Being wound down — new signups already closed - AWS-focused — less useful for non-AWS codebases - Provider choice limited to Claude via Bedrock/CLI; no OpenRouter or broad model-agnosticism

9.3 JetBrains AI Assistant

What it is: JetBrains AI Assistant is integrated into IntelliJ IDEA, PyCharm, WebStorm, GoLand, and all other JetBrains IDEs. It provides chat, completion, and an autonomous multi-step agent called **Junie** — worth naming explicitly, since it is JetBrains' direct answer to agentic coding assistants rather than a generic "agentic features" bundle.

Pricing:

- **AI Free** — $0; 3 AI Credits per 30 days
- **AI Pro** — Individual $10/month with 10 credits; Business $20/month with 20 credits
- **AI Ultimate** — Individual $30/month with 35 credits; Business $60/month with 70 credits
- **AI Enterprise** — custom

All Products Pack, dotUltimate, and some IDE licences can include AI entitlements. AI Credits are the quota unit; top-up credits are available on eligible individual/business plans and expire after 12 months. External models and agents can also be used without a JetBrains AI subscription, but feature coverage differs.

LLM models and agents: JetBrains documents current Claude, Gemini, OpenAI, and other model choices, plus BYOK and local providers such as Ollama, LM Studio, and llama.cpp. Junie, Claude Agent, Codex, Copilot, and ACP-compatible agents can be used from JetBrains IDEs; exact availability depends on IDE version and account.

Strengths: - Best choice if you are already heavily invested in JetBrains IDEs - Deep IDE integration (refactoring, inspections, test generation) - Junie provides genuine autonomous multi-step agent capability, not just chat - Local model support now confirmed, closing a prior gap

Weaknesses: - Junie is less proven than Cursor's agent mode or Cline on complex multi-file tasks - Pricing and entitlements vary between standalone AI plans, IDE bundles, and external-agent paths, adding cost-tracking overhead - JetBrains IDEs remain heavier than VS Code

9.4 Tabnine

What it is: Tabnine is an AI coding assistant with a long history (one of the first serious AI coding tools), historically focused on fast, high-quality inline completions. It has since pivoted towards agentic workflows.

Pricing (materially higher than before — the old ~$12/month Pro tier was sunset in 2025):

- **Code Assistant** — $39/user/month
- **Agentic Platform** — $59/user/month; adds MCP integration, autonomous workflows, CLI access
- **Enterprise** — custom, with self-hosted model option

Ownership and LLM provider: Tabnine announced its acquisition by Tricentis on 30 July 2026. Tabnine uses its own models plus third-party models (Anthropic, etc.); provider-hosted inference can add variable token charges and a handling fee, while Enterprise still supports deployment options for data sovereignty.

Strengths: - Very fast completions (latency-optimised) - Enterprise self-hosted option — best for organisations with strict data policies - Agentic Platform tier now offers MCP integration and CLI access, closing the gap with agent-first tools

Weaknesses: - Pricing jumped sharply ($12 → $39/$59/month) — no longer the budget completions option it once was - Agentic capability is newer and less proven than Cursor, Cline, or Claude Code - Being overtaken by Cursor/Windsurf on raw completion quality

9.5 Supermaven — discontinued, folded into Cursor

What it is: Supermaven was a VS Code-focused autocomplete tool with a very large context window for completions (300k tokens), notable for speed and accuracy.

Status: Supermaven is no longer an independent product. It was **acquired by Anysphere (Cursor's parent company) in November 2024**, not by Cursor itself as a product — Anysphere absorbed the team and technology, and Cursor has integrated Supermaven's completion engine into its tab-completion feature. If Supermaven was your tool of choice, Cursor is its direct successor; there is no standalone Supermaven product to evaluate separately.

10 Cross-Cutting Analysis

10.1 Provider Flexibility Analysis

A critical factor for Claude Code users concerned about API costs is whether an alternative tool is model-agnostic — allowing you to route requests to cheaper models, local models, or whichever provider has the best current pricing.

10.1.1 OpenRouter Compatibility

OpenRouter is a unified API gateway that provides access to hundreds of models from dozens of providers under a single API key, with pay-per-token pricing and no monthly subscription.

Tool	OpenRouter Support	Notes
Aider	Yes (verified)	Full setup at aider.chat/docs/llms/openrouter.html; via litellm.
Cline	Yes (verified)	Listed on openrouter.ai/works-with-openrouter.
OpenCode	Yes (verified)	75+ providers incl. OpenRouter; do not use the prohibited Claude Pro/Max plugin route.
Continue.dev	Yes	Via OpenRouter provider in config.yaml.
Goose	Partial	Reachable via custom API endpoint config; not an explicitly named provider in Goose's own docs.
Amp (Sourcegraph)	Partial	Supported provider-key configuration; OpenRouter is not a named first-class integration in the current docs.
OpenAI Codex CLI	Partial	ChatGPT subscription is the native path; local via `--oss`; OpenRouter reachable via API-compatible config, not a first-class integration.
Plandex	Partial	Via OpenAI-compatible endpoint.
SWE-agent	Partial	Typically direct Claude/OpenAI; OpenRouter-capable but not the primary pathway.
OpenHands	Partial	Via litellm provider abstraction; not a first-class named provider in docs surfaced this round.
GitHub Copilot	Partial	Local BYOK exists in several clients; enterprise-managed custom models are a separate path. OpenRouter and feature coverage depend on client, plan, and policy.

Tool	OpenRouter Support	Notes
GitHub Copilot CLI	Partial	Official BYOK covers only the `openai`, `azure`, and `anthropic` provider types (`openai` = any OpenAI Chat Completions-compatible endpoint); OpenRouter is not named as a supported provider, so it is reachable only unofficially via the generic `openai`-compatible path.
Grok Bot	No	Managed product; do not confuse xAI API access to Grok models with Grok Bot's product surface.
Zed AI	**Yes (was Partial)**	Explicitly listed alongside 10+ other providers (Bedrock, DeepSeek, Copilot, LM Studio, Mistral, Ollama, Vercel).
Google Antigravity 2.0	No	Current plans do not offer BYOK/custom endpoints for extending account quotas; third-party model availability is product/plan-specific.
Cursor	No	BYO key for direct providers only.
Windsurf (→ Devin Desktop)	No	Cognition-managed inference; not OpenRouter-oriented.
Devin	No	Cognition-managed inference with plan quotas and on-demand credits.
Gemini CLI	No	Locked to Gemini models; individual consumer access moved to Antigravity CLI.
Amazon Q Developer	No	Locked to Amazon/Anthropic via Bedrock.
AWS Kiro IDE	No	AWS-native model routing.
JetBrains AI Assistant	No	Tied to JetBrains' own model rotation, not user-configurable OpenRouter.
Tabnine	No	Own models plus a fixed provider list.

Conclusion: For OpenRouter flexibility, the open-source CLI tools (Aider, Cline, OpenCode, Continue.dev, and Goose) remain the clearest options. Amp supports provider keys but does not document OpenRouter as a named first-class integration. Zed AI also lists OpenRouter among its supported providers. OpenAI's Codex CLI supports local/OSS models via `--oss`, but OpenRouter is not its primary path. GitHub Copilot now has local BYOK in several clients and enterprise-managed custom models, but availability and feature cover-

age vary by client and plan. Cursor, Devin Desktop, Devin, and Grok Bot remain managed products rather than OpenRouter-style routing layers; Grok Bot's cross-application computer use should not be confused with provider flexibility.

10.1.2 Local LLM Support (Ollama / LM Studio)

Running models locally eliminates API costs entirely, but not hardware, electricity, or operational costs. Quality and hardware fit vary by model, quantisation, context length, and runtime; validate the exact model/workload combination before standardising on it.

Tool	Ollama	LM Studio	Notes
Aider	Yes	Yes	Via litellm.
Cline	Yes	Yes	Native provider options.
OpenCode	Yes	Yes	Provider-agnostic architecture.
Continue.dev	Yes	Yes	localhost:11434 default.
Goose	Yes	Partial	On-device philosophy is core; MCP-configurable local providers.
Amp (Sourcegraph)	No	No	Local CLI and self-hosted Orb execution are available, but inference remains on hosted providers; self-hosted execution is an Enterprise deployment option, not local model inference.
OpenAI Codex CLI	Yes	Yes	Via `--oss` flag; documented local providers are Ollama and LM Studio.
Plandex	Partial	Partial	Via OpenAI-compatible API config.
OpenHands	Yes	Yes	Via litellm; Agent Canvas can be self-hosted.
GitHub Copilot	Yes	Yes	Local BYOK is documented for several clients, including Ollama and Microsoft Foundry Local integrations; coverage varies by client and feature.
GitHub Copilot CLI	Yes	No	Copilot CLI's own BYOK docs name Ollama explicitly as a local endpoint example under the `openai` provider type; LM Studio is not named there (it appears only in the separate Copilot app's BYOK docs) — reachable, if at all, only via the generic OpenAI-compatible endpoint path, unendorsed.

Tool	Ollama	LM Studio	Notes
JetBrains AI Assistant	**Yes (was not listed)**	Yes	Ollama, LM Studio, llama.cpp confirmed on IDEs v2025.1+.
Zed AI	Yes	Partial	Ollama explicitly supported.
Windsurf (→ Devin Desktop)	No	No	Current product documentation confirms local-machine agent execution, not local LLM inference.
Devin	No	No	Devin Cloud uses Cognition-managed inference; do not conflate local execution with local model support.
Grok Bot	No	No	Hosted computer-use product; xAI API model access is a separate path.
Cursor	No	No	Cloud models only.
Gemini CLI	No	No	Gemini-hosted models; individual consumer access moved to Antigravity CLI.
Amazon Q Developer	No	No	AWS/Bedrock-locked.
AWS Kiro IDE	No	No	AWS-native.
Tabnine	No	No	Cloud/on-prem/air-gapped deployment options exist, but not consumer local-model tools like Ollama.
Google Antigravity 2.0	No	No	Google-managed models and quotas; the current plans page says no BYOK or custom endpoint for additional quota.

Best tools for local LLMs: Aider, Cline, OpenCode, Continue.dev, Goose, and JetBrains AI Assistant remain the clearest options for local model use. OpenAI's Codex CLI supports local models through `--oss` with documented Ollama and LM Studio providers. Amp's local CLI is not evidence of local inference; its current documentation describes hosted models and provider keys. Devin Desktop's local-machine execution should not be treated as local model support without explicit provider documentation.

For checkpoint choices and memory sizing, see Model Alternatives for Coding Agents, espe-

cially Deployment Fit. The tables above describe provider compatibility; model quality and hardware feasibility require separate evaluation.

10.2 Feature Comparison Matrix

The matrix is a shortlist rather than a complete catalogue; it spans 17 representative tools and is split below into five groups for legibility. Products and model surfaces change faster than this table, so read it with the dated product profiles and model section.

Group 1: Claude Code, Aider, OpenCode, Cline

Feature	Claude Code	Aider	OpenCode	Cline
Interface	CLI	CLI	TUI	VS Code / CLI
Agentic file editing	Yes (full)	Yes (full)	Yes (full)	Yes (full)
Multi-file context	Yes	Yes	Yes	Yes
Shell execution	Yes	Yes	Yes	Yes
Web search	Yes	No	Partial	Yes
Browser use	Via MCP/WebFetch‡	No	No	Yes
MCP support	Yes (native)	No	Yes	Yes
Custom hooks	Yes (skills+hooks)	No	Plugins / skills / hooks	Skills / hooks / plugins
OpenRouter	No	Yes	Yes	Yes
Local LLMs	No	Yes	Yes	Yes
Open source	No	Yes (Apache-2.0)	Yes (MIT)	Yes (MIT)
Git integration	Yes	Yes (auto-commit)	Yes	Yes
Cost model	Max plan or API	API only	API or subscription	Hosted provider, API, or local
Self-hosted option	No	N/A	N/A	N/A

Group 2: Continue.dev, Goose, Gemini CLI, OpenHands

Feature	Continue.dev	Goose	Gemini CLI	OpenHands
Interface	VS Code / JetBrains	CLI + Desktop	CLI	Web / CLI
Agentic file editing	Yes (partial)	Yes (full)	Yes (full)	Yes (full)
Multi-file context	Yes	Yes	Yes (1M tokens)†	Yes
Shell execution	Yes (agent mode)	Yes	Yes	Yes (sandboxed)
Web search	No	Yes	Yes	Yes
Browser use	No	No	No	Yes
MCP support	Partial	Yes (native)§	Yes	Yes (config)
Custom hooks	config.yaml	Extensions	Settings	Limited
OpenRouter	Yes	Partial	No	Partial
Local LLMs	Yes	Yes	No	Yes
Open source	Yes (Apache-2)	Yes (Apache-2)	Yes (Apache-2)	Yes (MIT)
Git integration	Yes	Yes	Yes	Yes
Cost model	API only	API/provider keys	Enterprise, API, or Vertex; consumer status disputed¶	API only
Self-hosted option	N/A	N/A	N/A	Yes

Group 3: Cursor, Windsurf, Codex CLI, Amp

Feature	Cursor	Windsurf	Codex CLI	Amp
Interface	IDE	IDE (now Devin Desktop)	CLI	CLI + Cloud
Agentic file editing	Yes (full)	Yes (full)	Yes (full)	Yes (full)
Multi-file context	Yes	Yes	Yes	Yes (agent context)

Feature	Cursor	Windsurf	Codex CLI	Amp
Shell execution	Yes (yolo mode)	Yes	Yes	Yes
Web search	Yes	Yes	Yes	Yes
Browser use	No	No	No	No
MCP support	Yes	Yes	Yes	Yes
Custom hooks	MCP / skills / hooks	Limited	Plugins / skills / hooks	Plugins / skills / hooks
OpenRouter	No	No	Partial	Partial
Local LLMs	No	No	Yes	No
Open source	No	No	Yes	No
Git integration	Yes	Yes	Yes	Yes
Cost model	$20-200/mo + API	Free / $20 / $200 / Teams	Bundled ChatGPT	Free / $20 Individual (orb minutes) / Teams+Enterprise pooled credits
Self-hosted option	No	No	No	Partial: self-hosted Orbs on Enterprise

Group 4: Devin, Grok Bot, Copilot

Feature	Devin	Grok Bot	Copilot
Interface	Cloud / Web UI	Cloud desktop/mobile app	IDE + Cloud
Agentic file editing	Yes (full, sandboxed)	Yes (computer-use)	Yes (full)
Multi-file context	Yes	Yes (persistent cloud computer)	Yes (Enterprise)
Shell execution	Yes (sandboxed)	Yes (cloud computer)	Yes
Web search	Yes (agentic browser tool)‖	Yes (browser/search)	Yes

Feature	Devin	Grok Bot	Copilot
Browser use	Yes	Yes	No
MCP support	Yes	Yes (connectors/custom MCP)	Yes
Custom hooks	Limited	Routines / skills / Auto Review	Limited
OpenRouter	No	No	Partial
Local LLMs	No	No	Yes
Open source	No	No	No
Git integration	Yes	Yes (via cloud computer)	Yes
Cost model	Free / $20 / $200 + usage	Eligible plan + separate Bot usage	$10-100/mo indiv.
Self-hosted option	No	No	No

Group 5: Kiro, Antigravity

Feature	Kiro	Antigravity
Interface	IDE + CLI + Web	Desktop + CLI + IDE integrations
Agentic file editing	Yes (full)	Yes (full)
Multi-file context	Yes	Yes
Shell execution	Yes	Yes
Web search	Yes	Yes
Browser use	No	Yes
MCP support	Yes (full)	Yes
Custom hooks	Hooks + steering + skills	Hooks + skills + plugins
OpenRouter	No	No
Local LLMs	No	No
Open source	No	No
Git integration	Yes	Yes
Cost model	$20-200/mo (credits)	Free (individuals)
Self-hosted option	No	Partial (SDK/enterprise paths)

Product details vary by plan or surface. † Gemini CLI's 1M-token context window is a model-level feature; product quotas still apply. ‡ Claude Code's browser access is via MCP servers (e.g. Playwright) or WebFetch, not a built-in browser. § Goose's MCP support is a confirmed core architectural pillar with documented extensions and broad MCP ecosystem compatibility. ¶ Google's transition announcement and current Gemini CLI authentication pages conflict on consumer access; verify the sign-in path before standardising on it. Grok Bot's browser, MCP, and computer-use capabilities are product-level features; do not infer equivalent capabilities for the xAI API from this row. ‖ Devin's web search is not a separately branded feature — it is the agentic cloud browser tool proactively looking up documentation and solutions during a task. This is distinct from "Devin Search," a separately documented codebase-search feature; do not conflate the two.

10.3 Extensibility Deep Dive

One of Claude Code's most powerful features is its extensibility system: Skills (reusable, discoverable workflow packages that can be invoked directly or selected by the agent), hooks (commands that fire on events like tool calls or session start), and MCP servers (external tools exposed via a standard protocol). This combination allows you to build a highly personalised, automated coding workflow. How do the alternatives compare?

10.3.1 Claude Code's Extension Architecture

For context, Claude Code's three-layer system works as follows:

1. **Skills** — Markdown files stored in `~/.claude/skills/` that define reusable workflows. When invoked as `/skill-name`, their content is loaded and executed. Skills can include checklists, decision trees, and instructions for specific types of tasks (debugging, code review, PR creation).

2. **Hooks** — Shell commands configured in `settings.json` that fire on specific Claude Code events. For example: run linting before a commit, post a notification when a session ends, validate that tests pass after edits. Hooks are executed by the Claude Code harness, not by the AI — this makes them reliable and deterministic.

3. **MCP Servers** — External processes exposing tools via the Model Context Protocol. Claude Code can call these tools as part of its reasoning (read from a database, query an external API, access browser dev tools, etc.). Any MCP server in the ecosystem works.

10.3.2 How Alternatives Compare

Aider: No equivalent to skills or hooks. No MCP support. Configuration via `.aider.conf.yml` covers model settings and preferences but not custom workflows. *Least extensible of the serious alternatives.*

OpenCode: MCP support confirmed. Its current plugin system can add custom tools, hooks, and integrations, and its configuration supports agent skills. The model and file formats differ from Claude Code, so portability is not automatic, but OpenCode now belongs in the serious extensibility comparison. *Mid-range extensibility, with a stronger automation story than the previous edition credited it with.*

Cline: MCP support confirmed. Current Cline adds project rules and skills, lifecycle hooks, plugins, schedules, and ACP support around the Plan/Act workflow. Its configuration is not compatible with Claude Code's skills or hook event schema, so migration still requires adaptation. *One of the closest alternatives for programmable local workflows.*

Continue.dev: The "blocks" system provides a plugin-like architecture for adding new context providers and tools. Config-driven customisation is deep. No hook system. MCP support via tool configuration. *Good for IDE users, weaker hook/automation story.*

Goose: MCP is a core architectural pillar, not a bolt-on — Goose's extension system is built directly on MCP, with documented extension support and broad MCP ecosystem compatibility. It is a strong conceptual match for Claude Code's MCP layer, though its format differs and its hook/skill semantics are not drop-in compatible. *Strong extensibility story.*

Gemini CLI: MCP, extensions, and hooks are documented. Individual consumer access is disputed across Google's current pages and the transition announcement; Antigravity CLI is the announced migration target and has skills, hooks, subagents, and plugins. Enterprise/API users should verify which features are present in their Gemini CLI release.

OpenHands: MCP support confirmed, with three transport types (SSE, Streamable HTTP, stdio) — a proxy approach (SuperGateway/FastMCP) is recommended over direct stdio for reliability. Configuration is required; MCP tools are not auto-discovered. Limited hook/automation equivalent otherwise. *Weakest extensibility story among the actively-developed alternatives.*

Grok Bot: Supports built-in connectors and custom MCP connectors, alongside skills, routines, Bot handoffs, and approval rules. MCP is optional rather than foundational: the Bot can use a browser and hosted computer directly when a service has no MCP server. Its routines and Auto Review controls are product-specific and are not compatible with Claude

Code hooks or other agents' skill formats.

10.3.3 The AGENTS.md Open Standard

Alongside MCP, a second cross-cutting convention has emerged that matters for anyone comparing extensibility stories: **AGENTS.md**, an open standard for project-level agent instructions. It was released in August 2025 and donated to the Linux Foundation's Agentic AI Foundation in December 2025. OpenAI's current announcement says it has been adopted by more than 60,000 open-source projects and agent frameworks, including Amp, Codex, Cursor, Devin, Factory, Gemini CLI, GitHub Copilot, Jules, and VS Code. This is an adoption claim from OpenAI, not an independent census. Claude Code itself can consume it through `CLAUDE.md` imports.

The distinction from MCP matters: MCP standardises *tool/capability* portability (how an agent reaches external systems); AGENTS.md standardises *project-instruction* portability (how an agent learns a repo's conventions, build steps, and constraints). They're complementary, not competing — a tool can support one, both, or neither. For anyone currently maintaining a pile of per-tool rule files (`.clinerules`, `.cursorrules`, `CLAUDE.md`), AGENTS.md is worth tracking as a potential single source of truth that many tools can read natively, reducing the duplication of instructions across tools.

10.3.4 What This Means for Migration

If you have heavily invested in Claude Code's extensibility system (skills, hooks, MCPs), a complete migration requires rebuilding your workflow on whatever alternative you choose. The good news: MCP server investments are often portable to any MCP-compatible tool (Cline, OpenCode, Gemini CLI, Goose, OpenHands, Amp, Antigravity, and Kiro), but transports, authentication, roots, sampling, approvals, environment variables, and client-specific behaviour still need testing. OpenCode, Cursor, Cline, Antigravity, and Kiro now have their own skills/hooks/plugin surfaces, but formats and event semantics differ. The hooks system is still the hardest part to reproduce faithfully — alternatives require their own plugin model, shell wrappers, or git hooks. If your per-tool instruction files (rather than MCP servers) are the bigger migration cost, look at consolidating onto AGENTS.md — it is now adopted across tens of thousands of projects, and Claude Code can import it into `CLAUDE.md`.

10.4 Cost Analysis

If you are a heavy Claude Code user currently on a Max plan, what would the same usage cost on each alternative? Note first that "unlimited usage" is not an accurate description

of the Max plan: Max 5x ($100/month) and Max 20x ($200/month) both carry a five-hour session limit and a separate weekly limit, with further caps and model-specific effects possible. Anthropic's live help page is the authority for the current limits; do not convert them into a fixed message count because message capacity depends on model, context, and workload.

10.4.1 Defining "Heavy Usage"

For this analysis, heavy usage means: - ~4-8 hours of active coding per day - Typical session: long context (50k-200k tokens input), many file edits, running tests - Estimated: ~100-200 million input tokens per month, 10-20 million output tokens per month

These are rough estimates — actual token consumption varies enormously by workflow and model. The worked examples below use the midpoints (~150M input / ~15M output tokens per month).

10.4.2 API Cost Estimates (Direct Provider, as of 7 October 2026)

Pricing changes frequently; treat these as ballpark figures using current published rates, not guarantees.

Anthropic Claude Sonnet 5.5:

- Check the current Anthropic pricing for the exact input, output, caching, and long-context rates before calculating a monthly estimate.
- Recompute the worked heavy-usage example from those published rates; do not carry forward the former Sonnet 5 price or the resulting ~**$450/month** estimate.
- *A Max plan may still be good value for genuinely heavy users, but compare its current caps and included usage with the current metered rates rather than relying on this historical example.*

Pricing note: pricing is volatile and model-specific. Verify Anthropic's current pricing page immediately before comparing providers; this refresh intentionally avoids carrying forward the superseded Sonnet 5 price claim.

Anthropic Claude Haiku 4.5:

- Input: $1/million tokens
- Output: $5/million tokens
- Heavy usage estimate: approximately **$225/month**
- *Cheaper and faster than the Sonnet tier in many workloads, but verify current tier pricing and caps before treating that as a purchasing recommendation.*

OpenAI GPT-5.6 Terra (current balanced tier):

- Input: $2/million tokens
- Output: $12/million tokens
- Heavy usage estimate: (150M x $2 + 15M x $12) / 1M approximately **$300 + $180 = ~$480/month**
- *GPT-5.6 Sol is the flagship at $4/$20 per million; GPT-5.6 Luna is the cost-sensitive tier at $0.20/$1.20. The model and price selected materially change the result.*

At the same 150M input / 15M output workload, the corresponding estimates are approximately **$900/month for Sol** ($600 + $300), **$480/month for Terra** ($300 + $180), and **$48/month for Luna** ($30 + $18), before caching or any subscription allowance.

Google Gemini 3.1 Pro Preview (paid API tier):

- <=200k context: $2.00 input / $12.00 output per million tokens
- >200k context: $4.00 input / $18.00 output per million tokens
- Heavy usage estimate: approximately **$480/month** at <=200k context, approximately **$870/month** above it
- *Gemini 3.5 Flash ($1.50/$9.00) has a separate API free tier with reduced quotas, but heavy usage as defined here will exceed it. Gemini CLI's individual Google-account path moved to Antigravity CLI on 18 June 2026.*

DeepSeek V4.1 Flash (`deepseek-flash`; illustrative direct-API rates; gateway rates differ):

- Context: 1M tokens; maximum output 384K; tool calls, Responses API, and vision supported
- Input: $0.15/million off-peak or $0.30/million peak (cache-hit input is lower)
- Output: $0.60/million off-peak or $1.20/million peak
- Heavy usage estimate: **~$32/month off-peak or ~$63/month peak** at the stated workload
- *Still dramatically cheaper than frontier models. DeepSeek retired the V4 Flash model on September 10 and now routes the old name to V4.1 Flash; rates vary by provider, peak/off-peak schedule, cache status, and gateway markup.*

Local model via Ollama or LM Studio:

- Cost: $0 in API charges, plus hardware and electricity
- Hardware requirement: depends on model, quantisation, context length, and runtime; validate the actual workload rather than relying on a generic VRAM threshold

10.4.3 Per-Tool Cost Summary

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Aider + Claude Sonnet 5	$0 tool + API	~$450/mo ($2/$10 standard pricing)	N/A	Illustrative calculation for the stated 150M input / 15M output workload.
Aider + Claude Haiku 4.5	$0 tool + API	~$225/mo	N/A	Cheaper same-vendor fallback; the "reduce cost, stay on Claude" scenario.
Aider + DeepSeek `deepseek-flash` (OpenRouter)	$0 tool + API	Provider/model-dependent	N/A	Current gateway pricing can differ from direct DeepSeek pricing and may include cache or peak/off-peak rates.
Aider + a low-cost OpenRouter model	$0 tool + API	Provider/model-dependent	N/A	Use the provider's live rate card; do not treat a single gateway quote as universal.
Aider + Ollama (local)	$0	$0	GPU hardware cost only	Choose a currently supported local coding model and validate its quantisation/context fit.
OpenCode + DeepSeek `deepseek-flash`	$0 tool + API	~$32–$63/mo	N/A	Uses DeepSeek V4.1 Flash direct rates for the stated off-peak/peak workload before any gateway markup.
OpenCode (ChatGPT-native)	$0 tool + ChatGPT subscription	Bundled — see Codex CLI row (same OpenAI subscription tiers apply since the Jan 2026 OpenAI partnership)	N/A	New row.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Cline + OpenRouter (model varies)	$0 tool + API	Varies by provider/model	N/A	Route cheaply via OpenRouter; use current provider pricing.
Amp (Sourcegraph)	Free (Hobby, pay-as-you-go) or $20/mo (Individual)	Individual $20/mo includes 45,000 minutes of orb time; Teams and Enterprise are pooled-credit, no fixed per-seat platform charge (as of 2026-09-12)	No	Enterprise supports self-hosted Orbs in AWS, Google Cloud, or Azure, but the Amp control plane and model traffic remain hosted. No markup over provider API prices for individual/non-enterprise plans; orb compute is metered separately by the minute with auto-pause.
Gemini CLI (individual)	Not applicable	Not applicable after the 18 June 2026 consumer transition	N/A	Use Antigravity CLI for individual Google-account access. Gemini CLI remains available through enterprise/API/Vertex paths.
Gemini CLI (teams/API)	Varies	Google Developer Program, AI Studio, and Vertex AI paths differ	N/A	Choose based on identity, privacy, quota, and billing requirements.
Google Antigravity 2.0	$0 individual tier	$0 individual tier with weekly limits; Google AI Pro/Ultra raise limits; enterprise is consumption-priced	No	Direct migration target for individual Gemini CLI users; exact limits and model availability vary.
Cursor Pro	$20/mo	$20/mo (usage-based credit pool, not a hard cap; heavy users may need Pro+/Ultra)	N/A	Old “500 fast requests/month” framing is gone; credit-pool billing since mid-2025.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Cursor Pro+	$60/mo	$60/mo (larger credit pool)	N/A	New tier.
Cursor Ultra	$200/mo	$200/mo	N/A	New tier.
Devin Desktop (formerly Windsurf) — Pro	$20/mo	$20/mo	No documented local inference	Current product name and pricing; local-machine execution is not local model inference.
Devin Desktop — Max	$200/mo	$200/mo	No documented local inference	Current tier.
Devin Desktop — Teams	$80/mo base + $40/seat	$80 + $40/seat	No documented local inference	Current tier.
Devin (cloud agent, Pro)	$20/mo	$20/mo plus on-demand credits at API pricing	N/A	Current plan; daily and weekly usage allowances apply.
Devin (cloud agent, Max)	$200/mo	$200/mo plus on-demand credits at API pricing	N/A	Current power-user plan with a larger weekly allowance and no daily cap.
Devin (cloud agent, Teams)	$80/mo minimum + $40/full seat	Shared on-demand credits; full seats include their own allowance	N/A	Replaces the legacy ACU/Core and $500 team-plan descriptions.
Grok Bot	Eligible SuperGrok / Cursor / Teams plan	Plan-dependent plus separate Bot usage allowance	N/A	Beta product; access, quotas, and billing vary by plan and change quickly.
GitHub Copilot — Individual Free	$0/mo	$0 (2,000 completions/mo cap)	N/A	New tier detail.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
GitHub Copilot — Individual Pro	$10/mo	$10 + variable consumption after included AI credits	N/A	AI-credit consumption varies by model and workload; code completions have a separate allowance.
GitHub Copilot — Individual Pro+	$39/mo	$39 + overage ($70 credits included)	N/A	New tier, includes premium models (Claude Opus access).
GitHub Copilot — Individual Max	$100/mo	$100 + overage ($200 credits included)	N/A	New tier.
GitHub Copilot — Business	$19/user/mo	$19/user/mo + policy-controlled model catalogue	N/A	Unchanged from old doc.
GitHub Copilot — Enterprise	$39/user/mo	$39/user/mo	N/A	Unchanged from old doc.
GitHub Copilot CLI	Bundled — inherits Copilot plan tiers above	Same	N/A	No standalone pricing found; open question whether it requires a specific plan tier.
Amazon Q Developer — Free	$0/mo	$0 (50 agentic requests/mo cap)	N/A	New row.
Amazon Q Developer — Pro	$19/user/mo	$19/user/mo (10,000 inference calls, ~1,000 user requests)	N/A	Also flag the sunset (new signups closed May 15 2026, support ends April 30 2027, Kiro is successor).
AWS Kiro — Free	$0/mo	$0 (50 credits)	N/A	New row.
AWS Kiro — Pro	$20/mo	$20 (1,000 credits) + $0.04/credit overage	N/A	New row.
AWS Kiro — Pro+	$40/mo	$40 (2,000 credits) + overage	N/A	New row.
AWS Kiro — Pro Max	$100/mo	$100 (5,000 credits) + overage	N/A	New row.
AWS Kiro — Power	$200/mo	$200 (10,000 credits) + overage	N/A	New row.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
JetBrains AI — Free	$0/mo	$0 (3 credits/30 days)	N/A	Some IDE bundles include AI entitlements; external models/agents are also available.
JetBrains AI — Pro	$10/user/mo individual; $20 business	Included 10/20 credits per 30 days; eligible top-ups	N/A	AI Credits are the quota unit; bundle and organisation rules vary.
JetBrains AI — Ultimate	$30/user/mo individual; $60 business	Included 35/70 credits per 30 days; eligible top-ups	N/A	AI Credits are the quota unit; bundle and organisation rules vary.
Tabnine — Code Assistant	$39/user/mo (annual)	$39/user/mo plus provider-dependent inference/handling charges	N/A	Replaces old doc's "~$12/month Pro" — that tier was sunset in 2025.
Tabnine — Agentic Platform	$59/user/mo (annual)	$59/user/mo plus provider-dependent inference/handling charges	N/A	Adds MCP integration, autonomous workflows, and CLI access.
Zed AI — Personal	$0/mo	$0 (2,000 edit predictions/mo)	N/A	Replaces old doc's vague "free tier and credits system."
Zed AI — Pro	$10/mo	$10 + token overage at API list price +10% ($5 tokens included)	N/A	New precise figure.
Zed AI — Business	$30/user/mo	$30/user/mo	N/A	New tier.
OpenAI Codex CLI	Bundled in ChatGPT plan	Free ($0), Go ($8/mo), Plus ($20/mo), Pro ($100/mo) — plan allowance and purchasable credits vary by tier; local `--oss` mode has no API charge	Hardware, electricity, and administration costs still apply to local `--oss` (Ollama/LM Studio)	Usage limits and included credits change by plan.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
OpenHands	$0 tool + API	Varies by chosen provider/model	N/A	Docker overhead is a hardware/time cost, not a monthly fee.

10.4.4 Cost Migration Strategy

Scenario 1: Keep Claude-quality results, accept higher costs Use Aider or Cline with direct Anthropic API. Cost: roughly $450/month for the heavy-usage workload defined here at Sonnet 5's current standard rate. This is still the worst-case scenario financially — you lose the Max plan subsidy, though the gap is narrower than it was under 2025 Sonnet pricing.

Scenario 2: Reduce cost with model diversity Use Aider or OpenCode with OpenRouter, routing to a current low-cost model for routine tasks and Claude/GPT-5.6 for complex ones. Cost depends on the selected provider, model, and routing strategy; expect a quality trade-off on simpler models.

Scenario 3: Go local for routine work, cloud for hard tasks Use Aider or Cline with Ollama or LM Studio and a current local coding model, then escalate to a cloud API for complex multi-file refactors. Cost is near zero for routine work, plus occasional API costs for hard tasks.

Scenario 4: Fixed-cost IDE subscription Move to Cursor (Pro $20/month, Pro+ $60/month, or Ultra $200/month) or Devin Desktop/Windsurf (Pro $20/month, Max $200/month). You lose terminal-native workflow; usage pools and on-demand usage mean the effective monthly cost is not necessarily fixed.

Scenario 5: Free-for-individuals alternative Antigravity CLI is the announced first-party migration path for individual Google-account users and has a $0 tier with weekly quotas. Google's current transition announcement conflicts with some Gemini CLI authentication pages, so test the live sign-in path before making it a dependency. API-key and Vertex use have their own quota, billing, and data-use terms.

11 Recommendations

11.1 If you want the closest CLI experience to Claude Code

Primary: Aider — mature, reliable, git-native, OpenRouter-supported, local LLM support. Start here. The absence of MCP and hooks is a real limitation if you've invested heavily in those, but the core editing workflow is the most Claude Code-like of any open-source tool.

Secondary: OpenCode — MCP support and broad provider flexibility (75+ connected providers) in a terminal tool with a very large user base. One caveat that changes its positioning: older bundled Anthropic plugin pathways are no longer bundled in current releases, while ChatGPT subscriptions work natively. If part of OpenCode's appeal to you was Claude-adjacency through older plugin flows, verify your current setup before committing.

Worth evaluating if you want mainstream backing or hybrid local/cloud: OpenAI's Codex CLI is a major CLI-native competitor, bundled into ChatGPT plans from Free through Enterprise, with local-model support via `--oss` for Ollama and LM Studio. Amp offers a hybrid local-CLI-plus-cloud-Orbs model with $20/$200 subscriptions or pay-as-you-go billing; model inference and Orb compute are separate meters.

11.2 If you want the best IDE-integrated alternative

Primary: Cline — model-agnostic, OpenRouter-supported, browser use, MCP-compatible, Plan/Act mode, and now available beyond VS Code through JetBrains and ACP-compatible editors, plus a full CLI. The most capable open-source agentic IDE/CLI tool available. Note: Roo Code, a 2024 fork of Cline that grew its own multi-agent following, was archived in May 2026; most of its userbase returned to Cline, reinforcing Cline's position here.

Secondary: Continue.dev — best-in-class local LLM support and deep IDE integration across VS Code and JetBrains simultaneously, and as of September 2025 it also ships a headless `cn` CLI for scriptable, non-interactive use — useful if you want IDE-first but occasionally need a scriptable escape hatch.

Commercial option: Cursor (Pro $20/month, Pro+ $60/month, Ultra $200/month) — if you want the best polished commercial experience and are comfortable with a VS Code-derived workflow. Note the terminology shift: "Composer" is Cursor's own in-house model, not the agent mode — autonomous execution is called "Agent mode." Current Cursor supports multiple parallel agents, but exact concurrency and branch limits are plan-dependent; it remains OpenRouter- and local-model-incompatible.

11.3 If cost is the primary constraint

Free option: Antigravity CLI has an official $0 individual tier authenticated with a Google Account. It is the simplest zero-subscription starting point in this survey, though weekly quotas and Google's model ecosystem still matter.

Ultra-cheap option: Aider or OpenCode with a current low-cost model via OpenRouter. DeepSeek V4.1 Flash is one candidate, but quote its current provider/model rate card at the time of purchase; use `deepseek-flash`, not the retired `deepseek-chat`, `deepseek-reasoner`, or legacy V4 Flash names.

Zero API cost: Any Ollama- or LM Studio-compatible tool plus a local coding model — hardware and latency depend on the model, quantisation, context, and workload.

A caution on Devin: Devin's $20/month Pro plan includes usage allowances, but extra work is purchased as on-demand credits at API pricing. That is clearer than the legacy ACU model, but it still means heavy users can spend beyond the subscription. Devin competes on autonomy and capability, not guaranteed low cost.

11.4 If you need persistent cross-application or asynchronous work

Primary: Grok Bot — if the task crosses websites, inboxes, documents, and business applications and can continue while you are away. Its persistent cloud computer and computer-use interface can reach systems that lack clean APIs or MCP servers. It is a poor fit when local repository control, local models, self-hosting, or strict no-cloud handling is the main requirement.

Coding-focused alternatives: Devin remains the better comparison for GitHub-centred autonomous software engineering, while OpenHands is the better comparison when you want an open-source, self-hostable cloud-agent stack. Claude Code remains the better fit for interactive terminal-native work.

11.5 If MCP server investments are critical

Tools with confirmed MCP support: Claude Code (native), OpenCode, Cline, Gemini CLI, Antigravity, Goose (MCP is a core architectural pillar with documented extensions and broad ecosystem compatibility), OpenHands (config, multiple transports), Amp, Kiro, and Grok Bot (connectors and custom MCP, alongside direct computer use). Of these, OpenCode, Cline, and Kiro have the strongest current combination of agentic execution and extensibility; Goose remains worth a closer look when on-device execution and MCP portability matter. Grok

Bot is the better fit when MCP is unavailable but browser or desktop computer use can bridge the gap.

11.6 If you need extensibility similar to Claude Code's skills+hooks

The honest answer: no alternative is drop-in compatible with Claude Code's skills+hooks system. The closest approximations: - **Skills equivalents:** Cline skills, OpenCode skills, Kiro skills, AGENTS.md-style instruction files, and custom system prompts - **Hooks equivalents:** Cline hooks, OpenCode hooks, Kiro hooks, Git hooks, shell wrappers, and CI/CD tooling — event names and permissions differ - **MCP equivalents:** Cline, OpenCode, Gemini CLI, Goose, OpenHands, and Amp all support MCP — your server investments are portable - **Instruction-file equivalents:** if per-tool rule files (rather than MCP servers) are your bigger migration cost, AGENTS.md is now adopted across tens of thousands of projects and is a better consolidation target than any single vendor's format

11.7 Migration strategy

Rather than a hard switch, a practical migration path:

1. **Set up Aider** as a Claude Code complement today. Get comfortable with it. It is free to try with Haiku (cheap) or Ollama (free).
2. **Test OpenCode** — its MCP support and provider flexibility make it a strong candidate for a full Claude Code replacement. Treat the old Claude Pro/Max plugin route as unsupported; use an Anthropic API key or another documented provider path.
3. **Test your free-tier fallback** — Antigravity CLI is free for individuals within product quotas. Run your own representative workload before relying on that allowance for daily work.
4. **Invest in OpenRouter** — get an API key if its routing breadth is useful. It reduces model lock-in, but adds a gateway dependency and does not automatically migrate you to cheaper models; select and re-evaluate models explicitly.
5. **Protect your MCP investments** — build MCP servers in preference to tool-specific plugins wherever possible. MCP compatibility is growing across the ecosystem. **Do the same for AGENTS.md** if you maintain per-tool instruction/rule files (`.clinerules`, `.cursorrules`, `CLAUDE.md`) — it is now adopted across tens of thousands of projects and is a lower-maintenance way to keep project instructions portable than maintaining one file per vendor.

12 A Concrete Multi-Model Alternative Setup

The survey above helps you choose a tool. It does not tell you how to run a real repository when no single model is best at every task. A useful alternative to Claude Code is a small, explicit routing policy: keep the harness stable, assign models to roles, and make escalation visible in the command or configuration. That gives you model diversity without hiding important cost, privacy, and quality decisions behind automatic routing.

This is an operating design, not a claim that these products or models are interchangeable. The model, provider, pricing, and policy examples below reflect a **18 September 2026** research checkpoint. Model IDs, prices, quotas, authentication methods, and tool support can change independently. Replace every example ID with one confirmed in the live provider catalogue before adoption.

12.1 The recommended default: OpenCode plus explicit model tiers

For this multi-provider, MCP-heavy design, start with OpenCode. Its MCP support, project configuration, local-model options, and broad provider catalogue make it a better centre of gravity than Aider when one workflow needs several providers and tool integrations. This is narrower than the single-tool recommendation above: use Aider as the minimal git-native fallback when you want a small editing loop and do not need MCP or an IDE planning surface.

OpenCode can select a main model and a smaller model explicitly. A project configuration can also describe an OpenAI-compatible local endpoint, but it must not become a place to store credentials:

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "openrouter/deepseek/deepseek-v4-pro-0813",
  "small_model": "openrouter/z-ai/glm-5.3-flash",
  "provider": {
    "openrouter": {
      "models": {
        "deepseek/deepseek-v4-pro-0813": {},
        "z-ai/glm-5.3-flash": {},
        "qwen/qwen3-coder-next": {}
      }
    },
    "ollama": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "Ollama (local)",
      "options": { "baseURL": "http://localhost:11434/v1" },
      "models": {
        "<local-model>": { "name": "Local coding model" }
      }
    }
```

```
  }
}
```

These are concrete examples, not permanent defaults. Authenticate through OpenCode's credential flow, environment variables, or the provider's credential store. Do not put an API key in `opencode.json`, commit it in a repository, or assume that an OpenAI-compatible endpoint provides reliable tool calls, long context, structured output, or reasoning controls. OpenCode's provider, configuration, and model documentation are the relevant setup references. The exact provider route is part of the experiment: pin a provider when a long tool-use session needs stable latency, or permit fallbacks when availability matters more than reproducibility.

12.2 Provider accounts, credits, and data boundaries

Create one separate API key for this coding harness, give it a small spending limit, and keep it in the shell environment or the harness credential store. Do not use a personal, unlimited production key while evaluating models. The following are practical provider choices for the open-weight roster:

Provider	Account and credential setup	Good use in this design	Data-retention position
OpenRouter (recommended default)	Create an account, buy prepaid credits (the current minimum top-up is $5 and unused credits can expire after 365 days), create a project/key, and set a daily or monthly key limit. Use `OPENROUTER_API_KEY`; Pi also supports `/login openrouter`.	One endpoint for DeepSeek, GLM, Qwen, Kimi, Mistral, and other open-weight models; model and provider fallbacks; one usage view.	OpenRouter says it does not retain prompts unless you enable prompt logging, but the selected inference endpoint has its own policy. Enable ZDR and `data_collection: deny`, and inspect the endpoint list before sending sensitive code. Automatic fallback can otherwise move a request to a different provider.
DeepSeek direct	Create a DeepSeek API account, top up its balance, and export `DEEPSEEK_API_KEY`. The direct API has its own model aliases and peak/off-peak pricing.	Lowest-cost direct access to DeepSeek V4 Pro and Flash when price and model-specific throughput matter more than a single gateway.	DeepSeek's privacy policy says it collects prompts and may use personal data for research, development, and foundation-model training, retaining it as needed for those purposes. Do not send confidential repositories without an approved organisational basis.

Provider	Account and credential setup	Good use in this design	Data-retention position
Z.ai direct	Create an API account/key and store `ZAI_API_KEY`; verify the current GLM API endpoint and model ID in the account documentation.	GLM 5.3/Flash when you want a direct commercial relationship, a different provider failure domain, or a second review route.	Z.ai's API DPA states that customer content is not stored on its servers for the API service, while other account and service data may be retained and processing is generally in Singapore. Confirm the applicable contract and region before relying on this for regulated data.
Qwen / Alibaba Model Studio (DashScope)	Create a region-specific Alibaba Model Studio account, activate the service, create a key, and use `DASHSCOPE_API_KEY`. Billing, endpoints, quotas, and model names vary by region; coding plans and metered API billing are separate products.	Qwen3 Coder Plus/Flash/Next when you want the model owner's endpoint, regional routing, or a direct price comparison against OpenRouter.	Treat the region's current Alibaba terms and data controls as authoritative. Do not assume that a China-region free quota, an OpenRouter endpoint policy, and a US/EU Model Studio account have the same retention or training terms.

Provider	Account and credential setup	Good use in this design	Data-retention position
Mistral direct	Activate Mistral Studio, create a key, and export `MISTRAL_API_KEY`. Free mode is suitable for a smoke test; paid usage uses API credits/pay-as-you-go. Use the EU endpoint when regional processing is required and supported.	Devstral and Mistral's open-weight coding models, especially when EU processing, direct support, or predictable regional routing matters.	Mistral's paid API policy describes up to 30 rolling days of retention for abuse monitoring unless ZDR is enabled; ZDR requires an eligible paid organisation and approval. Training opt-out and ZDR are separate controls.
Together AI	Open an account, purchase prepaid credits, create a scoped key, and use `TOGETHER_API_KEY`. Together currently requires at least $5 in credits for API keys and supports auto-recharge.	Shared or dedicated inference for Qwen, DeepSeek, and other open checkpoints when throughput, hardware, or provider-specific pricing beats the gateway.	Treat retention as provider-specific; the current public material does not establish a ZDR guarantee sufficient for sensitive repositories. Review the current contract before use.

Provider	Account and credential setup	Good use in this design	Data-retention position
Fireworks	Create an account and `FIREWORKS_API_KEY`; the hosted serverless API is a direct endpoint, while dedicated deployments have separate billing.	Fast serverless testing of Qwen, DeepSeek, Kimi, and other open models, with a different provider failure domain from OpenRouter.	Fireworks documents no persistent prompt/generation logging unless you opt in, but its Responses API stores conversations by default for up to 30 days; set `store=false` and verify the endpoint mode.
Groq or Cerebras	Create a project/key and use `GROQ_API_KEY` or `CEREBRAS_API_KEY`. Groq has free/developer limits and paid monthly caps; Cerebras offers a free key and separate paid channels.	Very fast supplementary routes for bounded work, triage, and review; coverage is narrower than OpenRouter.	Groq documents no customer-content retention by default but can retain reliability/abuse logs unless ZDR is enabled. Cerebras states that prompts, outputs, and request/response logs are not retained, while operational and billing metrics remain. Verify current terms before treating either as a regulated-data route.

Provider	Account and credential setup	Good use in this design	Data-retention position
NVIDIA NIM or local serving	Use a Developer Program key for prototyping or an `NGC_API_KEY` to pull self-hosted containers; local Ollama, vLLM, SGLang, or llama.cpp needs no hosted credit account.	Shared GPU inference or full local control for Qwen, DeepSeek, Devstral, GLM, and other checkpoints.	Self-hosting can keep prompts local, but the NVIDIA hosted developer service has non-production restrictions. Local serving does not prevent telemetry or external MCP/editor integrations from exporting data.
Local Ollama, vLLM, SGLang, or llama.cpp	Download the exact checkpoint, configure the local server, and use a loopback base URL. No hosted API credits or provider key are required.	Devstral Small 2, Qwen3-Coder-30B-A3B, or GLM-4.7-Flash for repositories that must stay on the workstation.	The inference request can remain local, but telemetry, extensions, model downloads, remote MCP servers, and editor integrations can still send data elsewhere. Audit the whole path.

Use OpenRouter as the starting point unless you have a clear reason to use a direct provider. It reduces account and integration work while retaining model choice, but it adds a gateway and does not erase the upstream provider's data policy. Set a modest initial balance—OpenRouter's performance guidance suggests keeping roughly $10–20 available to avoid low-balance interruptions—then set a lower key or workspace budget such as $25/month for the pilot. Use a separate key for experiments, an allowlist containing only the model/provider combinations you intend to test, and automatic top-up only after the first month's actual usage is understood.

For sensitive work, prefer a local runtime. If hosted inference is necessary, use Mistral's approved ZDR path or OpenRouter with ZDR, `data_collection: deny`, an allowed regional endpoint, and a provider allowlist. DeepSeek direct is a price recommendation, not a privacy

recommendation. The model weights being openly available says nothing about what a hosted endpoint logs or trains on.

The credential flow should be explicit:

```
# OpenRouter: use a scoped key, never a key committed to the repository.
export OPENROUTER_API_KEY="..."

# Direct-provider alternatives; set only the one being tested.
export DEEPSEEK_API_KEY="..."
export ZAI_API_KEY="..."
export MISTRAL_API_KEY="..."

# Smoke-test the selected route without asking an agent to edit files.
opencode --model openrouter/deepseek/deepseek-v4-pro-0813
```

The shell exports are examples; use a password manager, `direnv` with a non-committed file, or the harness's credential store in real use. Rotate a key after exposing it in a terminal capture, CI log, screen recording, or issue. A `401` usually means the wrong key/provider or an expired login; `402` usually means missing credits or a provider account limit; a `429` usually means rate limiting; and a tool-call or context error can mean that the selected endpoint does not implement the model capability advertised by the catalogue.

For OpenCode's current v1 configuration, enter the interactive client and use `/connect` for the providers it knows, then `/models` to select a model. It stores credentials outside the repository; `opencode auth list` helps separate an authentication failure from a model or endpoint failure. Keep the project `opencode.json` limited to provider and model metadata. OpenCode's v2 configuration is a different schema, so do not copy a v2 `providers`/`settings` example into a v1 `provider`/`options` file.

Pi's provider flow is similar but more explicit. Run `pi`, use `/login openrouter` for the browser or headless authorisation flow, then `/model`; or export `OPENROUTER_API_KEY`. Pi also maps direct keys such as `DEEPSEEK_API_KEY`, `MISTRAL_API_KEY`, `ZAI_API_KEY`, and `DASHSCOPE_API_KEY` when the corresponding provider is configured. Stored credentials live in `~/.pi/agent/auth.json` with restrictive permissions and take precedence over environment variables. Local Ollama, vLLM, and llama.cpp providers normally belong in Pi's `models.json` and OpenCode's custom-provider configuration; bind them to `127.0.0.1` and use the model's required tool parser.

12.3 Rough cost planning

For a simple planning estimate, assume one month contains 2 million input tokens and 500,000 output tokens. At the 18 September 2026 snapshot, the model-only arithmetic is approximately:

Model route	Approximate monthly token cost at that workload
DeepSeek V4 Pro 0813	$2.31
GLM 5.3	$3.71
Qwen3 Coder Next	$0.64
DeepSeek V4.1 Flash	$0.60
GLM 5.3 Flash	$0.28
Qwen3 Coder 30B A3B	$0.28

This is a budgeting example, not a bill. Agent sessions often resend repository instructions, file contents, tool results, and summaries, so a long debugging session can consume many times the tokens of the visible conversation. Reasoning tokens, cache-hit rates, provider routing, retries, minimum charges, and any gateway or BYOK fee also change the result. Start with a $10–25 cap, record actual usage by model and provider, and raise the limit only when the task pilot justifies it. Local inference replaces token billing with hardware, electricity, setup, and maintenance costs; count failed runs and operator time rather than calling it free.

12.4 Model allocation by task

Use a role-based policy that you can inspect and override. A wrapper script or shell alias is enough; automatic model switching that the user cannot see is not.

Role	Open-weight starting choice	When to use it
Plan and architecture	DeepSeek V4 Pro 0813	Hard design, migrations, threat modelling, and changes whose wrong abstraction will be expensive
Implement and debug	GLM 5.3 or Qwen3 Coder Next	Multi-file edits, tests, debugging, and tool-driven repository work; use GLM when reasoning depth matters and Qwen when coding throughput and cost matter

Role	Open-weight starting choice	When to use it
Routine work	GLM 5.3 Flash or Qwen3 Coder Next	Summaries, small edits, documentation, test-output triage, and short refactors
Sensitive or offline work	Devstral Small 2 24B or a Qwen3-Coder checkpoint via Ollama, LM Studio, vLLM, or llama.cpp	Data that must stay on the workstation, after testing the checkpoint's context and tool behaviour
Independent review	Kimi K3 or GLM 5.3 through a different provider route	High-risk changes where correlated model mistakes are costly; use the expensive Kimi route selectively
Asynchronous execution	OpenHands or Devin in an isolated workspace	Explicitly bounded work that can run away from the interactive session and still receive human approval before merge

At the 18 September 2026 checkpoint, OpenRouter listed DeepSeek V4 Pro 0813 at about \$0.66 per million input tokens and \$1.98 per million output tokens, GLM 5.3 at about \$1.00/\$3.41, GLM 5.3 Flash at about \$0.075/\$0.25, and Qwen3 Coder Next at about \$0.12/\$0.80. DeepSeek V4.1 Flash was another inexpensive long-context option at about \$0.15/\$0.60 on OpenRouter; its direct API documents Flash/Pro input-cache-miss rates of \$0.15/\$0.66 off-peak or \$0.30/\$1.32 peak, and output rates of \$0.60/\$1.98 off-peak or \$1.20/\$3.96 peak. Compare the actual route rather than copying one number. Qwen3 Coder 30B A3B Instruct was about \$0.07/\$0.27 and is a good bounded-edit or structured-output tier. Treat these as list-price snapshots, not a cost estimate: cached input, provider choice, routing, retries, reasoning tokens, and output length change the bill. GLM 5.3 always reasons and supports selectable reasoning effort; Qwen3 Coder Next is a non-thinking coding model, so do not compare their raw token prices without comparing the work they complete.

For local work, benchmark Devstral Small 2 24B, Qwen3-Coder-30B-A3B, and GLM-4.7-Flash first. Devstral Small 2 is an Apache-licensed coding checkpoint; Qwen3-Coder-30B-A3B is the smaller coding specialist; and GLM-4.7-Flash is a compact agent model. Serve them through Ollama for a quick single-user trial, or vLLM/SGLang when reliable tool parsing and shared serving matter. Qwen3-Coder and Devstral need their model-specific tool/parser

configuration in some runtimes. Record the exact checkpoint, quantisation, runtime, context budget, and tool-call settings; a model that fits in memory may still fail at long-context retrieval or tool-call formatting. Hardware, quantisation, context length, and tool-call support determine whether a local model is useful. Keep it in the routine/offline tier until it passes representative repository tasks; local availability is not evidence of frontier parity.

The routing policy should remain legible in the workflow:

```
# Examples from the September 18, 2026 checkpoint; confirm live IDs first.
opencode --model openrouter/deepseek/deepseek-v4-pro-0813
opencode --model openrouter/z-ai/glm-5.3
opencode --model openrouter/qwen/qwen3-coder-next
opencode --model openrouter/qwen/qwen3-coder-30b-a3b-instruct
opencode --model ollama/<local-model>

# Aider is the deliberately smaller fallback.
aider --model openrouter/qwen/qwen3-coder-next \
      --weak-model openrouter/z-ai/glm-5.3-flash
```

12.5 Setup: CLI-first

Install OpenCode and connect the providers you have approved. Put repository-wide instructions in `AGENTS.md`; keep `opencode.json` focused on model and provider adapters. Begin with DeepSeek V4 Pro for hard tasks, GLM 5.3 Flash for cheap routine work, and Qwen3 Coder Next or Qwen3 Coder 30B A3B for the coding-focused middle tier. Add a local endpoint after verifying context and tool behaviour. Direct DeepSeek, Mistral, Z.ai, Qwen, Together, Fireworks, NVIDIA NIM, and Groq endpoints are alternatives to OpenRouter when you need a different billing, region, or data-retention boundary; the same model name does not imply the same tool implementation. For a minimal Aider fallback, use a repository-level `.aider.conf.yml` with no secrets:

```
# .aider.conf.yml -- do not store secrets here
model: openrouter/qwen/qwen3-coder-next
weak-model: openrouter/z-ai/glm-5.3-flash
edit-format: diff
auto-commits: false
```

Aider's model metadata can describe context limits, costs, edit format, and a weak model when a provider's model is not already known to it. That is useful for reproducibility, but it is still local client configuration, not a guarantee that the remote model supports the advertised capabilities. Choose Aider when the small git-native loop is the requirement; do not describe it as an equivalent replacement for Claude Code's MCP, skills, and hooks architecture.

12.6 Setup: agentic editor

Use Cline when the editor is the primary surface and you want Plan/Act review, browser and MCP access, and a visual diff loop. Configure OpenRouter with GLM 5.3

for difficult Act-mode changes, Qwen3 Coder Next for ordinary implementation, and GLM 5.3 Flash for bounded tasks. Cline can use the same provider and model IDs through OpenRouter, but that does not imply behavioural equivalence: test the full harness/provider/model/context/tool-schema/reasoning-settings tuple, and remember that approval semantics remain Cline-specific.

Use Continue.dev instead when local Ollama or LM Studio inference and IDE context matter more than autonomous tool breadth. It is a lower-risk editor choice for sensitive code if the endpoint, telemetry, context providers, and extensions are all reviewed. In either editor, keep the plan reviewable, require approval before writes with external consequences, and do not assume that importing an `AGENTS.md` file reproduces Claude Code's full instruction or hook behaviour.

12.7 Repository configuration and security boundaries

Treat the harness, provider, model, repository, and external tools as separate trust boundaries. MCP standardises tool access; it does not standardise permissions, data retention, or approval policy. A gateway such as OpenRouter is a provider-boundary decision, not merely a convenient billing layer.

Keep secrets in the provider credential store or environment variables. Never commit them in `opencode.json`, `.aider.conf.yml`, editor settings, or `AGENTS.md`. Keep local-only data on a local model or an approved endpoint, and remember that prompts, retrieved files, tool results, and telemetry can cross the boundary even when the model itself is not hosted by the tool vendor.

Require explicit review before destructive commands, migrations, credential access, external messages, and merges. The model can propose an action; the harness, shell policy, branch protections, and CI must enforce authorisation. A practical starting permission matrix is:

Role	Read/write	Shell	Browser/MCP	Destructive actions	Merge
Plan and architecture	Read; no writes	No	No by default	No	No
Implement and debug	Read/write branch	Approved commands	Approved servers only	Confirmation required	No
Routine work	Read/write scoped files	Tests and formatters only	No by default	No	No
Sensitive / offline	Local only	Local allowlist	Approved local	No	No
Independent review	Read-only fresh checkout/diff	Tests in a sandbox	No by default	No	No
Asynchronous agent	Isolated worktree	Least privilege	Explicitly approved connectors	No automatic approval	No

Model selection must not silently expand these permissions. For OpenHands or Devin, distinguish execution isolation from data isolation: a Docker sandbox can still send source through a hosted model or MCP service, while Devin is a managed cloud boundary. Use a separate branch or worktree with least-privilege credentials and a clean handoff containing the task contract, diff, relevant source, tests, and unresolved questions. Human approval remains the integration gate.

12.8 A staged rollout and evaluation loop

Start with a pilot of roughly 20-30 representative tasks from the repository, including a new feature, multi-file refactor, failing-test diagnosis, documentation edit, and tool or MCP task. Run each task with the proposed frontier, fast, local, and independent-review routes where they apply. Record the harness and model IDs, provider, permissions, reasoning settings, context and quantisation, retries, successful completion, missed defects, false positives, review burden, latency, cost, tool failures, and data-boundary exceptions. A model that finishes quickly but creates a large review burden is not cheap in practice.

An independent review needs fresh context, not just a second opinion in the original conversation. Give the reviewer the task contract, diff, relevant source, test results, and unresolved

questions; measure both missed defects and false positives. A different provider reduces one source of correlated error, but it does not create statistical independence by itself.

Promote a model to a broader role only after it passes that task set with the permissions it will actually receive. Re-run the benchmark when a provider, model ID, quantisation, context setting, harness version, or major project instruction changes. Keep representative prompts and expected checks in the repository's normal evaluation workflow so the result is reproducible rather than anecdotal.

12.9 What this setup does not solve

Multiple models do not remove the need for good repository instructions, tests, observability, or human judgement. A second model can repeat the first model's mistake, especially when both rely on the same provider, training data, or misleading project context. OpenRouter does not make every provider equivalent, and a local endpoint does not make sensitive data safe if the harness or an MCP server sends it elsewhere. MCP portability still varies by transport, authentication, filesystem roots, approvals, and client behaviour; skills and hooks are not drop-in portable. `AGENTS.md` can consolidate instructions, but it does not standardise permissions or tool execution. An asynchronous agent does not become trustworthy merely because it works in the cloud or reports success.

The concrete recommendation is therefore: begin with OpenCode plus OpenRouter, use DeepSeek V4 Pro 0813 for the small number of hard tasks, GLM 5.3 or Qwen3 Coder Next for implementation, GLM 5.3 Flash for routine work, and Devstral Small 2 or a sized Qwen checkpoint locally when data must stay on the workstation. Keep Cline as the editor surface if that matches how you work, and retain Aider as the minimal fallback. Use a different provider route for high-risk review, and refresh model IDs, prices, quotas, authentication paths, and provider policies immediately before adoption.

13 Pi as an Additional Harness

Pi is worth adding to the comparison because it takes a different position from the full-featured alternatives above. It is a minimal terminal coding harness: the core provides the agent loop, a small set of file and shell tools, model selection, sessions, and a TUI, while extensions, skills, prompt templates, themes, and packages supply the workflow-specific parts. Pi's design deliberately leaves out several features that other harnesses bundle. That makes it useful as an additional, inspectable harness for experimentation and focused interactive work, but it does not make it a drop-in replacement for Claude Code.

This section describes Pi as documented on **18 September 2026**. The project has had package and repository-scope changes, so verify the current package name, release, provider catalogue, authentication path, and documentation before installing. The current official site uses the `@earendil-works/pi-coding-agent` package; older examples may use the `@mariozechner` scope.

13.1 What Pi adds to a multi-harness setup

Pi is a useful third surface when you want to hold the harness constant enough to compare models, but keep the harness itself easy to inspect and extend. Its interactive session supports switching models and thinking levels during a task, automatic session persistence, branching and forking, context compaction, and direct access to the repository. Its print/JSON modes support one-shot or machine-readable runs, while RPC mode and the TypeScript SDK make it possible to embed Pi in a wrapper or another application. The practical low-cost pairing is OpenRouter for hosted open-weight models and `llama.cpp`, Ollama, or another local provider for offline work; Pi's provider documentation includes OpenRouter, DeepSeek, Mistral, NVIDIA, Together, Fireworks, and custom compatible endpoints. A compatible endpoint is only a transport: Pi still needs the model's correct context, reasoning, developer-role, and tool-call settings.

That combination creates several practical roles:

- Use Pi for exploratory repository work where a low-friction terminal loop and mid-session model switching matter more than a built-in plan or approval workflow.
- Use Pi as a model-harness experiment: run the same task contract, repository revision, permissions, and acceptance checks through Pi and OpenCode or Claude Code, then compare accepted changes, recovery, latency, and cost.
- Use Pi as a custom workflow host when a small TypeScript extension can express the required tool, command, approval, or context behaviour more clearly than a large product configuration.
- Use Pi's print, JSON, RPC, or SDK surfaces for bounded automation, provided the caller supplies its own authorisation, timeout, logging, and merge gates.

Do not use Pi merely because it can call the same underlying model. A model's behaviour depends on the complete tuple of harness, provider, model ID, context policy, tool schemas, thinking settings, retries, and permissions. A successful Pi run does not prove that the same model will behave the same way in OpenCode, Cline, or Claude Code.

13.2 Installation and a first controlled run

The current official quick start offers an installer or a global npm installation. Pin the version in a reproducible environment where the harness itself matters, and inspect the package source before granting it access to sensitive repositories:

```
# Verify the current command and package version before installing.
npm install -g --ignore-scripts @earendil-works/pi-coding-agent
pi --version

# Start from a disposable branch or worktree.
pi
```

For the recommended open-weight path, run `/login openrouter`, add credits in the OpenRouter dashboard, select the OpenRouter provider, and choose a versioned model ID with `/model`. On a headless host, paste the final redirect URL or authorisation code as Pi documents. Alternatively, export `OPENROUTER_API_KEY`. Direct-provider examples are `DEEPSEEK_API_KEY`, `ZAI_API_KEY`, `MISTRAL_API_KEY`, and `DASHSCOPE_API_KEY`; the key, endpoint, region, and model catalogue must match. Do not copy an old OAuth or subscription workaround from a forum into a production workflow. A subscription login is an access path with its own terms; it is not evidence that every model or feature is available through every provider.

Pi's default loop gives the model `read`, `write`, `edit`, and `bash`; `grep`, `find`, and `ls` are available as additional read-only tools through tool options. Start with a bounded request and explicit checks:

```
pi -p "Inspect the repository instructions. Do not edit files. \
Report the test command, build command, and three likely failure points."
```

For a write task, create a branch or worktree first, give Pi the repository contract, require tests, and inspect the diff yourself. Pi runs with the permissions of the user who starts it, so the working directory and shell account are part of the authorisation boundary.

13.3 Project context, settings, and extensions

Pi can load `AGENTS.md` and `CLAUDE.md` context files, which makes it a useful consumer of portable repository instructions. Keep those files about repository invariants, commands, tests, and safety rules. Do not treat them as a permission system. Project-local Pi resources live under `.pi/` and can include settings, extensions, skills, prompts, and themes; global resources live below `~/.pi/agent/`.

For example, a project settings file can select a documented startup model and restrict the initial built-in tools:

```
{
  "defaultProvider": "openrouter",
  "defaultModel": "deepseek/deepseek-v4-pro-0813",
```

```
  "defaultThinkingLevel": "medium",
  "defaultTools": ["read", "grep", "find", "ls"]
}
```

The model ID is a 17 September 2026 starting point, not a permanent default. For a cheaper Pi profile, switch to `z-ai/glm-5.3-flash` for summaries and small edits, `qwen/qwen3-coder-30b-a3b-instruct` for structured bounded changes, or `qwen/qwen3-coder-next` for routine code work. For hard interactive work, compare `deepseek/deepseek-v4-pro-0813` with `z-ai/glm-5.3`; for offline work, select `Devstral-Small-2-24B-Instruct-2512`, `Qwen3-Coder-30B-A3B`, or `GLM-4.7-Flash` through a configured llama.cpp/Ollama/vLLM provider. Project settings and resources require a project-trust decision in interactive use. Trust controls whether Pi loads project-local resources; it is not a sandbox. Non-interactive modes need an explicit trust choice or a suitable `--approve`/`--no-approve` policy, and an unattended run must not inherit a permissive setting accidentally.

Extensions are TypeScript modules that can register custom tools, commands, UI, event handlers, and tool-call interception. They can implement approval prompts, policy checks, custom compaction, or an MCP adapter, but that flexibility moves security responsibility to the extension author. Pi packages can bundle extensions, skills, prompt templates, and themes from npm, Git, URLs, or local paths. Official documentation warns that extensions execute arbitrary code and skills can instruct the model to perform actions, so review and pin third-party packages before enabling them. Do not install an unreviewed package in a repository containing credentials or production access.

13.4 What Pi does not provide by default

Pi's minimal core intentionally does not provide built-in MCP, plan mode, subagents, permission popups, background bash, or a built-in to-do system. That is a design choice, not an unfinished claim of parity. You can add some of these through extensions, packages, scripts, tmux, or a wrapper, but the result becomes your workflow and must be tested as a new harness configuration.

The absence of built-in approval popups is the sharpest operational difference. For local experiments, a disposable worktree and container boundary may be enough. For a shared repository, add an approval extension or put Pi behind a wrapper that allowlists commands, blocks destructive operations, records tool calls, and stops before merge. Do not infer safety from the project-trust prompt: Pi's own security guidance says trust does not restrict what tools can do after the run is trusted.

The absence of built-in MCP also changes migration cost. Existing MCP servers are not

automatically available in Pi. An extension can bridge them, or a CLI tool with a stable interface can be invoked through `bash`, but transport, authentication, roots, result size, approval, and failure-recovery behaviour all need separate validation. If MCP is central to the workflow, OpenCode or Cline remains the more direct additional harness. The model choice does not repair this gap: GLM, DeepSeek, and Qwen can all emit tool calls, but Pi still owns the tool schema, approval policy, retries, and result handling.

13.5 Sessions and cross-harness handoff

Pi automatically saves sessions and supports continuing the most recent session, browsing older sessions, opening a session by path or ID, branching, forking, cloning, and compaction. This is valuable for interactive exploration, but Pi session files are not a portable continuation format for Claude Code, OpenCode, or Cline. A durable handoff should contain the repository revision, task contract, changed files, diff, tests and results, model/provider identifiers, unresolved questions, and any approval decisions. The shared state is the worktree plus written checkpoint, not the private transcript.

A practical handoff from Pi to the main harness looks like this: Pi explores the repository and records a short plan in the issue or checkpoint file; OpenCode or Claude Code implements the approved change in a dedicated worktree; a different provider receives the contract, diff, relevant source, and test results for review; CI and a human own the merge. The reverse direction works for investigation or a small follow-up, but do not carry hidden assumptions across the boundary.

13.6 How to evaluate Pi fairly

Add Pi as another row in the existing 20-30-task pilot, not as an anecdotal alternative. Use the same initial revision, task contract, context files, permission envelope, model/provider tuple, thinking level, retry policy, and acceptance checks. Record Pi's version, package scope, provider, model ID, context and session settings, extension/package versions, local runtime and quantisation where relevant, tool failures, latency, cost, accepted changes, missed defects, false positives, and human correction time.

Include at least one task that needs repository exploration, one multi-file implementation, one failing-test diagnosis, one documentation edit, one long-session continuation, and one tool-integration task. For the last two, test compaction, session resume, malformed tool results, and interruption recovery. If Pi needs an extension or wrapper to supply a capability that OpenCode has in its core, record that configuration as part of Pi's harness identity and measure its maintenance burden.

13.7 Recommendation

Add Pi when you value a small terminal core, explicit model switching, inspectable session state, or TypeScript-level customisation. Keep it beside OpenCode rather than underneath it: use OpenCode for the multi-provider/MCP-heavy default described above, Pi with DeepSeek V4 Pro or GLM 5.3 for difficult interactive work, Pi with GLM 5.3 Flash or Qwen3 Coder Next for cheap routine work, and Pi with Devstral Small 2 or Qwen locally for offline work. Use Cline for an editor-first Plan/Act surface and Aider for the minimal git-native fallback. Use a disposable worktree or container for initial trials, review extensions and packages as code, and never let Pi's model choice silently expand shell, file, network, or merge permissions.

14 References

First-party sources checked on **18 September 2026** for the current model, provider, pricing, and policy snapshot. Pi sources and setup documentation were checked on **18 September 2026**. Grok Bot sources were checked on **9 September 2026**. Model names, prices, quotas, and product boundaries can change independently; the date is part of each claim.

14.1 Model Documentation and Licences

- Anthropic model overview, Fable 5.1, and Fable 5.1 announcement
- OpenAI API models, GPT-5.6 announcement, and OpenAI gpt-oss introduction
- OpenAI gpt-oss-20b, gpt-oss-120b, OpenAI open-weight model guidance, and local Ollama guide
- Gemini API model catalogue, Gemini 3.1 Pro model card, and Gemini 3.8 Flash specifications
- Gemma model overview, Gemma 3 model card, and Gemma deployment guide
- xAI Grok 4.6 documentation
- Qwen3-Coder announcement, Qwen Code model providers, and Qwen3.8-27B model card
- Qwen3-Coder runtime guidance, Qwen3 Coder 30B A3B on OpenRouter, and Qwen3 Coder Next on OpenRouter
- DeepSeek V4.1 Flash model card, V4.1 release information, and Responses API
- DeepSeek V4 Pro 0813 on OpenRouter and DeepSeek V4 Pro model card
- Z.AI GLM-5 documentation and GLM-5.3-Flash release
- GLM 5.3 on OpenRouter, GLM 5.3 Flash on OpenRouter, and GLM-4.7-Flash model card

- Kimi model catalogue, Kimi K3 developer guide, Kimi K3 technical blog, and Kimi K3 model card and licence link
- Mistral Devstral offline-model guidance, Devstral release, Devstral 2 deprecation, and Mistral Medium 3.5
- Devstral Small 2 model card, deployment guidance, and Apache 2.0 licence
- Meta Llama developer resources and Llama model repository
- MiniMax M2, NVIDIA Nemotron, IBM Granite Code, and Microsoft Phi-4 mini

14.2 Model Evaluations and Practitioner Reports

- DeepSWE v1.1 common-harness leaderboard, methodology, and mini-SWE-agent harness
- Claude Code request for automatic model routing
- Codex long-task stall report
- OpenCode context-coherence discussion
- Claude Code community comparison of Claude, Codex, and Gemini
- Claude community discussion of Fable/Opus/Sonnet/Haiku role changes
- Engineering pitfalls in Claude Code, Codex, and Gemini CLI
- Cross-model LLM code review study

14.3 Tool and Pricing Documentation

- Anthropic Max plan and usage limits, Claude Code with Pro or Max, and Anthropic API pricing
- OpenAI Codex pricing, Codex model documentation, and Codex repository/current CLI distribution
- OpenAI API pricing: service tiers and short/long-context rates
- Gemini CLI to Antigravity CLI transition, Gemini CLI authentication, Gemini API pricing, Antigravity plans, Antigravity overview, CLI installation, and MCP documentation
- Amp pricing, self-hosted Orbs, documentation, skills and plugins, and plugin API
- Cline CLI reference, installation and supported surfaces, and OpenRouter provider
- OpenCode providers, configuration, models, developer providers, and repository
- OpenCode releases, v1-to-v2 configuration migration, and OpenCode authentication
- Goose repository, OpenHands MCP guide, and OpenHands MCP settings
- Cursor pricing and Cursor rules
- Devin Desktop, Cognition's Windsurf acquisition, Devin pricing, and self-serve billing
- Zed pricing/providers and Windows availability

- Kiro pricing, documentation, steering, hooks, and MCP
- JetBrains AI plans, supported models, and agents
- Tabnine pricing and acquisition announcements
- GitHub Copilot BYOK and models/pricing
- Amazon Q IDE end-of-support scope, DeepSeek API pricing, and OpenAI Agentic AI Foundation announcement
- OpenRouter credits and API keys, API-key limits, BYOK, ZDR, guardrails, and model fallbacks
- DeepSeek API pricing and top-ups and DeepSeek privacy policy
- Z.AI API quick start, Z.AI pricing, and Z.AI API privacy/DPA
- Alibaba Model Studio API-key setup, regional base URLs, and model pricing
- Mistral Studio API-key setup, API pricing, ZDR, privacy policy, and regional inference
- Together AI credits and billing, API-key setup, Fireworks pricing, and Fireworks data handling
- Groq models and pricing, Groq data controls, Cerebras authentication, and Cerebras data retention
- Aider configuration, repository, advanced model settings, and edit formats
- Pi documentation, quick start, providers, usage and CLI modes, settings and project trust, extensions, skills, sessions, RPC, SDK, and security
- OpenRouter works-with-openrouter, Continue Ollama guide, and AGENTS.md

14.4 Secondary Background

- Artificial Analysis coding-agent taxonomy
- Awesome CLI Coding Agents
- MorphLLM's Claude Code alternatives comparison